European Journal of Operational Research 331 (2026) 62–76

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/eor

Discrete Optimization

On location-routing formulations for the Hamiltonian p-median problem

Francisco Canas *, Luís Gouveia

Center for Mathematical Studies (CEMS.UL), Lisbon, Portugal

ARTICLE INFO

Keywords:
Combinatorial optimization
Integer linear programming
Location-routing
Hamiltonian p-median
Branch-and-cut

ABSTRACT

Given a positive integer p and a weighted undirected graph $G = (V, E)$, the Hamiltonian p-Median Problem (HpMP) on G is to find a minimum weight set of p elementary cycles partitioning V. We study extended formulations for this problem including node-depot assignment (NDA) variables, in addition to edge variables. These formulations are based on the selection of certain nodes as depots and thus can be viewed as formulations for location-routing problems. Known NDA formulations and valid inequalities are reviewed, and new extended formulations, including edge-depot assignment (EDA) variables, are presented. We relate EDA formulations with known NDA formulations and, from the former, derive new exponentially sized sets of constraints defined with the edge and NDA variables. Computational results show that the EDA formulations produce very strong lower bounds and are effective for instances with up to 100 nodes and large values of p, and that the branch-and-cut algorithms based on NDA formulations including (some of) the generalized constraints are competitive with (and often outperform) the current state of the art (which involves solving instances with up to 400 nodes). We also address and present computational results for a variant of the HpMP in which setup costs for depots are considered.

1. Introduction

We focus our study on location-routing formulations for the Hamiltonian p-Median Problem (HpMP), in which, given a complete undirected graph $G = (V, E)$ with weights d_{ij}, $\forall\{i, j\} \in E$ and an integer p, the objective is to determine the minimum total cost set of p elementary and vertex-disjoint cycles spanning all vertices of the graph. The designation "location-routing" for the formulations studied in this paper will be clarified later on.

The HpMP is related with other known routing problems. Among others, we mention i) the Travelling Salesman Problem (TSP) (see Applegate et al., 2006), which arises as a particular case of the HpMP if $p = 1$; ii) the Triangle Packing Problem (see Garey & Johnson, 1979), which arises as a particular case of the HpMP if $p = n/3$; iii) the 2-factor problem (see Cornuéjols & Pulleyblank, 1980), which arises as a relaxation of the HpMP, if p is not fixed.

Literature Review. The HpMP was first introduced by Branco and Coelho (1990) as a location-routing problem (see Belenguer et al., 2011; Laporte et al., 1983) embedding the p-Median and Travelling Salesman problems with application to various real-world problems, such as in determining the location of schools or other depots for industrial and commercial purposes. The typical application is illustrated by the assignment of p guards to n objects, with each guard cycling among the objects assigned to them. Applications also include maintenance/inspection vehicles for machines and cutting problems.

In the literature, two variants of HpMP are studied, depending on whether 2-cycles (that is, cycles with 2 nodes) are allowed or not. The variant not allowing 2-cycles, which this paper focuses on, has been studied extensively using different exact modelling approaches. Glaab and Pott (2000) and Gollowitzer et al. (2011) have developed formulations that include edge variables in addition to variables through which nodes and/or edges are assigned to cycles. In the latter, these formulations were also enhanced by symmetry-breaking (SB) constraints.

In Gollowitzer et al. (2014), the authors introduce new formulations (including some of the NDA formulations discussed in this paper), revisit known formulations for the HpMP and conduct polyhedral and computational studies to compare those formulations. Some of the formulations discussed in that work were then enhanced by Erdoğan et al. (2016), who have also developed a branch-and-cut algorithm, in addition to a new constructive heuristic and an Iterated Local Search (ILS) algorithm. Both the branch-and-cut algorithm and the ILS algorithm proved to be effective for instances of the HpMP with up to 100 nodes.

Marzouk et al. (2016) consider a different approach, presenting a branch-and-price algorithm based on a modification of a set-partitioning formulation first introduced by Branco and Coelho (1990). Compared with the state-of-the-art at the time, this algorithm was only effective for instances with large values of p. Bektaş et al. (2019) introduced a

* Corresponding author.
E-mail address: fmcanas@fc.ul.pt (F. Canas).

https://doi.org/10.1016/j.ejor.2025.10.023
Received 8 August 2025; Accepted 15 October 2025
Available online 27 October 2025
0377-2217/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

branch-and-cut algorithm for both variants of the problem (allowing and not allowing 2-cycles), solving some benchmark instances more effectively than previous methods.

More recently, some work has been done on the study of exact methods for the HpMP based on both compact (see Barbato et al., 2023) and non-compact formulations (see Barbato & Gouveia, 2024). The former will be discussed in more detail later. Regarding the latter, Barbato and Gouveia (2024) propose new constraints to prevent solutions with less than p cycles and also address the particular cases of the HpMP where $p = \lfloor n/3 \rfloor$, by proposing inequalities which are valid only for those cases. Barbato and Gouveia (2024) also develop and test a branch-and-cut algorithm, which was capable of solving instances with up to 400 nodes, outperforming state-of-the-art methods.

Motivation and Further Background. Let p^* denote the number of cycles in the optimal solution of the 2-factor relaxation and v^* its value. Then, it is clear that for any value of p, the optimal value $v(p)$ of the corresponding HpMP instance is such that $v(p) \geq v^*$, and if $p = p^*$, $v(p) = v^*$ holds. Furthermore, as discussed by Marzouk et al. (2016) and Bektaş et al. (2019), for most benchmark instances, v tends to decrease as p increases from 1 to p^*, and tends to increase as p increases from p^* to $\lfloor n/3 \rfloor$. In many instances, v is actually monotonously decreasing between 1 and p^* and monotonously increasing between p^* and $\lfloor n/3 \rfloor$. An immediate consequence of how v changes as a function of p is that, for values of $p > p^*$, constraints to prevent solutions with less than p cycles are more relevant than constraints to prevent solutions with more than p cycles, since, if $p > p^*$, there are few or no solutions with more than p cycles with a value lower than $v(p)$, and thus, those solutions cannot be optimal - indeed, for very large values of p, constraints to prevent solutions with more than p cycles can often be discarded without affecting the optimal value. Likewise, for instances with $p < p^*$, the opposite happens, and constraints to prevent solutions with more than p cycles become more relevant. Computational testing by Bektaş et al. (2019) corroborates these findings.

Motivated by these findings, Barbato et al. (2023) (and more recently, Barbato & Gouveia, 2024) have given more emphasis to constraints used to limit the number of cycles in any feasible solution to at least p, and thus, also proposed a variant of this problem in which feasible solutions can include more than p cycles (denoted by HpMP$_{\geq}$). This emphasis is a consequence of the fact that there seems to exist a better understanding of constraints to prevent more than p cycles due to there being similarities between these constraints and subtour elimination constraints for the TSP. This contrasts with the knowledge on constraints to prevent solutions with fewer than p cycles, which is negligible in comparison. Also, as explained earlier, such constraints become more relevant as p grows larger, and instances of this problem with large values of p have proven to be difficult to solve (see Barbato et al., 2023; Bektaş et al., 2019; Erdoğan et al., 2016). For this reason, we also focus on the HpMP$_{\geq}$. However, since we will also present computational results for the problem with exactly p cycles, we will also use a set of inequalities to prevent solutions with more than p cycles proposed (and has been shown to be quite effective) by Canas and Gouveia (2025).

Barbato et al. (2023) provides a theoretical and computational comparison of the Linear Programming (LP) relaxations of four classes of formulations, namely disaggregated and aggregated versions of node-cycle assignment (NCA) and node-depot assignment (NDA) formulations. The former are characterized by the inclusion of additional variables which assign each node to the cycle it belongs to (e.g., if node i is in the kth cycle, that node is assigned to the integer k), whereas the latter are characterized by the inclusion of additional variables which assign each node to the index of a node that is designated/selected as the depot of the cycle it belongs to (e.g., if node d is the depot of the cycle node i is in, then, node i is assigned to the integer d). These formulations can be viewed as formulations for the more general class of location-routing problems. In fact, as noted in earlier studies (see Bektaş et al., 2019), there is a strong connection between "path-elimination" constraints from multi-depot routing problems and constraints preventing solutions with less than p cycles in the HpMP, explaining the relevance of focusing on these type of models.

The results of this paper indicate that further research on formulations that lead to more efficient methods is needed, for instance, by developing formulations with stronger LP relaxations. The results obtained from the study with the four classes of formulations lead us to focus on the study of the disaggregated NDA formulations, namely extensions and generalizations which also include edge-depot assignment (EDA) variables. Since we will not focus on the aggregated formulations, we will, from now on, omit the "disaggregated" designation when referring to the disaggregated NDA formulations. Observe that although the EDA formulations include more variables compared to the NDA formulations, the NDA variables are easily written as linear transformations of the EDA variables, and thus, any formulation written with the NDA variables can be rewritten with EDA variables. Therefore, it is always possible to obtain formulations that include EDA variables whose LP relaxations are as strong as those of formulations that include only edge and NDA variables. In addition to this, the information of the EDA variables makes it easier to derive new intuitive (and potentially strong) valid inequalities leading to formulations that are stronger than currently known NDA formulations. Furthermore, in theory, the linear transformations relating the two sets of variables lead to the possibility of obtaining NDA formulations with a stronger LP relaxation than currently known ones. These new formulations result from projecting out the EDA variables from the set of feasible solutions of the LP relaxations of the EDA formulations. Several studies comparing edge/arc with node variables (or reformulating node into edge/arc based formulations) are known from the literature, which assess that edge/arc based formulations are stronger than node-based formulations (although the former also have more variables). One example is given by the well-known Single-Commodity Flow and Miller-Tucker-Zemlin formulations (see Gavish & Graves, 1978 and Miller et al., 1960 respectively) which arise in several routing problems. Many other examples are also taken from less topical problems, such as the tool-switching problem (see Catanzaro et al., 2015).

Contributions of the paper. This paper is the result of a study into how the LP relaxations of known formulations using NDA and standard edge variables (see Barbato et al., 2023; Erdoğan et al., 2016; Gollowitzer et al., 2014) can be strengthened by considering additional EDA variables. In particular, we investigate whether one can obtain polynomially sized formulations with EDA variables that imply exponentially sized sets of constraints defined in the space of the original edge and NDA variables, with emphasis on constraints to prevent solutions with fewer than p cycles. Our main theoretical findings can be described as follows: i) we define new formulations including EDA variables which produce stronger LP relaxation bounds; ii) we show that one of the polynomially sized formulations is equivalent, in terms of LP relaxations, to a NDA formulation including an exponentially sized set of constraints (that prevent solutions with less than p cycles) proposed in Erdoğan et al. (2016); iii) by strengthening the previous compact formulation with an additional (polynomially sized) set of valid inequalities, we obtain a new formulation that implies a larger set of valid inequalities that can be viewed as a generalization of the constraints from Erdoğan et al. (2016). The computational study shows that the stronger polynomially sized formulation produces very strong LP relaxation bounds and the reported computational times to obtain the optimal integer LP (ILP) solution show that it is competitive with the best branch-and-cut algorithms known from the literature for instances with up to approximately 100 nodes (especially if $p = \lfloor n/3 \rfloor$). For previously known benchmark instances with up to 400 nodes, a branch-and-cut algorithm based on a formulation that includes some of the generalized constraints proves to be more effective and is indeed competitive with the current state-of-the-art. We have also created a new set of more challenging instances for which none of the branch-and-cut algorithms based on exponentially sized formulations perform particularly well. Although the inclusion of the gener-

alized constraints does lead to notable improvements in lower bounds and computational times, the polynomial-sized formulation continues to demonstrate strong competitiveness in these instances.

Connections with enhancements for directed formulations. A similar enhancement, based on the use of arc-depot assignment (ADA) variables, has also been proposed by Bektaş et al. (2019) for the directed version of the HpMP. However, the study of EDA formulations differs from the study of ADA formulations. In particular, it is far from clear how to adapt the standard conservation constraints known from ADA (and other directed) formulations to EDA formulations. This is because the conservation constraints for ADA formulations rely on the direction of the arcs. However, edges are not directed, and thus a different, and less straightforward approach is required to develop valid EDA formulations. Bektaş et al. (2019) also present some results showing that the ADA formulation implies some node-based exponentially sized inequalities known from the literature, including a directed version of the aforementioned constraints proposed by Erdoğan et al. (2016). Concerning the undirected version of those constraints, we present an equivalence result, in contrast to the previously mentioned implication result. Moreover, based on what was explained before, the proof of the implication for directed constraints is not trivially adaptable for the undirected case. This equivalence result allows us to have a better understanding of the strength of the class of inequalities given by Erdoğan et al. (2016). Furthermore, we present the derivation of a new exponentially sized set of constraints that generalize constraints from Erdoğan et al. (2016).

Outline. The paper is organized as follows. In Section 2, we introduce the notation used throughout the paper. Section 3 includes a review of known formulations (Gollowitzer et al., 2014, Erdoğan et al., 2016, Barbato et al., 2023), as well as new (exponentially sized) sets of constraints to prevent solutions with more and less than p cycles. Results are also presented that relate known and new formulations. Some of these results can also be viewed as motivation for the current work. In Section 4, we present new formulations with EDA variables and relate some of the new formulations to other formulations discussed before. Furthermore, we present new sets of valid inequalities defined with the edge and NDA variables, which are obtained by projecting the polyhedra associated with the LP relaxations of the new formulations into the space of the edge and NDA variables. Section 5 includes a brief discussion of the applicability of the SB strategy to variants of this problem. Finally, Section 6 provides the results taken from a computational experiment to evaluate the efficacy of different exact algorithms based on the formulations discussed in the preceding sections. The results of algorithms for both the HpMP and HpMP$_{\geq}$ are given, and comparisons are also made with algorithms proposed by Erdoğan et al. (2016), Bektaş et al. (2019), Barbato et al. (2023) and Barbato and Gouveia (2024).

2. Notation

In this paper, some notation is considered to simplify the remainder of the paper. When naming formulations for the HpMP and HpMP$_{\geq}$, to help distinguish the formulations for these two variants, we add a "$_{\geq}$" at the end of the name of each formulation for the HpMP$_{\geq}$. Thus, if a formulation for the HpMP is denoted by A, the formulation resulting from removing constraints which prevent solutions with more than p cycles from A (valid for the HpMP$_{\geq}$) is denoted by A$_{\geq}$.

Furthermore, given some Mixed ILP (MILP) formulation F, we denote by ADM(F) the set of feasible solutions of F (note that any LP or IP is also a MILP), and by F$_L$ its LP relaxation (that is, the formulation resulting from relaxing the integrality constraints in F, replacing these with constraints enforcing lower and upper bounds on the variables). Moreover, consider two sets of variables x, z with p and q variables, respectively. Consider also the set $S \subset \mathbb{R}^{p+q}$. We define PROJ$_x$(S)= $\{x' \in \mathbb{R}^p : \exists z' \in \mathbb{R}^q$ with $(x', z') \in S\}$. In other words, PROJ$_x$(S) is the projection of the set S onto the subspace defined by the x variables.

Given an undirected graph $G = (V, E)$, where V is the set of nodes and E is the set of edges in that graph, and given two sets of nodes $S_1, S_2 \subseteq V$, we denote by $E(S_1, S_2)$ the set of edges $\{\{i, j\} \in E : i \in S_1, j \in S_2\}$. If $S_1 = S_2$, we denote this simply by $E(S_1)$. Finally, if any of the sets includes only one element i, we can represent it using only the index of the node instead of as a set of one element (so, for instance, we can write $E(i, S_2)$ instead of $E(\{i\}, S_2)$). Finally, given variables u_{ij}, $\forall\{i, j\} \in E$ and a set $P \subseteq E$, we define $U(P) = \sum_{\{i,j\}\in P} u_{ij}$. Likewise, given variables z^d_{ij}, $\forall\{i, j\} \in E$, $\forall d \in V$ and some set $P \subseteq E$, we consider $Z^d(P) = \sum_{\{i,j\}\in P} z^d_{ij}$.

3. A review of formulations from the literature

In this section, we review formulations described by Erdoğan et al. (2016) and Barbato et al. (2023) that are needed for the development of the remainder of the text. As in Barbato et al. (2023), we begin by describing a non-valid formulation using only edge and node variables upon which all other valid formulations for the HpMP and HpMP$_{\geq}$ described here are built. Using these two sets of variables, we introduce two new classes of constraints, one to prevent solutions with strictly more than p cycles and the other to prevent solutions with strictly less than p cycles. These two classes are based on two combined ideas: the use of variables to identify one node of each circuit as a special node (a "depot" node) and an SB strategy that selects the node with the lowest index to be the depot of each cycle. For preventing solutions with more than p cycles, we present a cycle-based set of inequalities, an enhanced clique-based set known from location-routing problems (see Laporte et al., 1983, Gollowitzer et al., 2014) and a further enhanced clique-based set presented in Canas and Gouveia (2025). To prevent solutions with fewer than p cycles, we present only a set of cycle-based constraints. The comparison of the enhancement from cycle-based to clique-based constraints in the first set with the yet unknown and far from clear similar enhancement for the latter set may be viewed as motivation for our study for deriving new and stronger constraints that prevent solutions with fewer than p cycles. This is described in Section 3.1.

Constraints known from the literature which include NDA variables and prevent solutions with fewer than p cycles are reviewed in Section 3.2.2, and valid inequalities that strengthen the LP relaxation bounds and also include the NDA variables are reviewed in Section 3.2.3.

3.1. The BASE formulation

Consider two sets of binary variables: variables u_{ij}, $\forall\{i, j\} \in E$, indicating if edge $\{i, j\} \in E$ is included in one of the p cycles, and depot variables y_i, $\forall i \in V$, which indicate whether node $i \in V$ plays the role of the depot of the cycle it is in. Consider, also, the following formulation, which we denote by **BASE**:

$$\text{Min.} \sum_{\{i,j\}\in E} d_{ij}u_{ij} \qquad \text{(OBJ)}$$

$$\text{s.t. } U(E(i, V \setminus \{i\})) = 2, \quad \forall i \in V \qquad \text{(DEG)}$$

$$\sum_{i\in V} y_i = p, \qquad (p\text{DEP})$$

$$u_{ij} \in \{0, 1\}, \quad \forall\{i, j\} \in E \qquad \text{(BIN-}u\text{)}$$

$$y_i \in \{0, 1\}, \quad \forall i \in V. \qquad \text{(BIN-}y\text{)}$$

The objective function corresponds to minimizing the sum of the weights of the edges which define the p cycles. Equalities (DEG) are the usual degree constraints stating that each node is adjacent to exactly two other nodes. Constraint (pDEP) indicates that the number of nodes that are depots is equal to p. Finally, constraints (BIN-u) and (BIN-y) define the u_{ij} and y_i variables as binary.

A feasible solution to the BASE formulation corresponds to a set of any number of node disjoint elementary cycles (with at least three nodes each) covering all nodes of the graph with exactly p activated depots,

which implies this is not a valid formulation for the HpMP or HpMP$_{\geq}$. A generic valid formulation for the HpMP can be obtained by including the generic constraints:

$\{\{i,j\} \in E : u_{ij} = 1\}$ contains at least p cycles. (GEN≥)

$\{\{i,j\} \in E : u_{ij} = 1\}$ contains at most p cycles. (GEN≤)

Observe that a generic formulation for the HpMP$_{\geq}$ only requires the generic constraints (GEN≥). As noted in previous studies (see Gollowitzer et al., 2014, Erdoğan et al., 2016, Bektaş et al., 2019), the main advantage of using formulations with depot variables is that these generic sets of constraints can be respectively restated as generic location-routing (see Bektaş et al., 2020) constraints as follows:

$\{\{i,j\} \in E : u_{ij} = 1\}$ contains no cycle with multiple depots. (GEN≥)

$\{\{i,j\} \in E : u_{ij} = 1\}$ contains no cycle with 0 depots. (GEN≤)

Note, however, that the different possibilities for selecting nodes to play the role of depots create many equivalent solutions, since there are now many solutions for the BASE formulation which represent the same set of cycles but differ only in the nodes that are considered to be depots. To address this issue, we follow the literature (see Erdoğan et al., 2016, Barbato et al., 2023) and consider the SB strategy mentioned at the start of Section 3 - in any given cycle, only the node with the lowest index is a depot. In other words, the constraints (GEN≥) and (GEN≤) can be further modified as follows:

$\{\{i,j\} \in E : u_{ij} = 1\}$ contains no cycle such that any of its nodes other than the lowest index node is a depot. (GEN≥)

$\{\{i,j\} \in E : u_{ij} = 1\}$ contains no cycle such that its node with the lowest index is not a depot. (GEN≤)

Since the SB strategy is always assumed throughout the rest of this paper, the designation of these generic constraints has not been modified. In the sets of constraints to be presented next, given a cycle C, we denote by $\min(C)$ the node with the lowest index in that cycle. The same notation holds for paths and sets. One first straightforward set of constraints that enforces the condition stated in generic constraints (GEN≥) is given by:

$$U(C) + Y(C \setminus \{\min(C)\}) \leq |C|, \qquad \forall \text{ cycle } C. \qquad \text{(CI≥)}$$

These constraints state that if some cycle C is in the solution, then, none of its nodes other than the one with the lowest index can be a depot. Although rather intuitive, an indirect proof of its validity will be given in later sections.

Concerning constraints (GEN≤), one straightforward set of constraints to enforce that condition is:

$$U(C) \leq |C| - 1 + y_{\min(C)}, \qquad \forall \text{ cycle } C. \qquad \text{(CI≤)}$$

These constraints state that if some cycle C is in the solution, then, the node with the lowest index in that cycle must be a depot (which implies $y_1 = 1$). (CI≤) illustrate what has been mentioned before regarding using knowledge from the TSP, since they can be lifted with the edge variables corresponding to edges linking non-consecutive nodes in the cycle (transforming the cycle into a clique), leading to:

$$U(E(S)) \leq |S| - 1 + y_{\min(S)}, \qquad \forall S \subset V. \qquad \text{(CC≤)}$$

These constraints were first proposed in Canas and Gouveia (2025). Observe that they can be viewed as a lifting of constraints known from the literature on location-routing and proposed by Laporte et al. (1983) (see also Gollowitzer et al., 2014), to prevent solutions with cycles with no depots:

$$U(E(S)) \leq |S| - 1 + Y(S), \qquad \forall S \subset V. \qquad \text{(wCC≤)}$$

The results of an extensive set of computational tests given in Canas and Gouveia (2025) comparing several sets of constraints to model (GEN≤) show that the constraints (CC≤), although not leading to the strongest LP relaxations, lead to the more efficient algorithms for the HpMP. For this reason, any valid formulation for the HpMP that is used in this paper will include this set of constraints. This will allow us to focus on the constraints that prevent solutions with fewer than p cycles.

Observe that for the (GEN≤) constraints, we have described the straightforward cycle-based inequalities as well as the corresponding enhanced version based on cliques. For the (GEN≥) constraints, we have described the cycle-based inequalities, but have not yet been able to provide any enhanced version. The point we want to emphasize is that inequalities for the case (GEN≤) are similar to (and generalize) inequalities known for the TSP (to prevent subtours) and there is a substantial amount of knowledge on these inequalities to help us develop inequalities for this case. However, for constraints (GEN≥), the current knowledge is not as substantial, we might even say that it is negligible compared to what is known for the first case. This analysis motivates the focus of this paper on using additional sets of variables and constraints to prevent solutions with less than p cycles. This, together with projection results, might be the right step towards finding good valid inequalities in the space of the u_{ij} and y_i variables.

3.2. NDA formulations

In this section, we study formulations that result from augmenting BASE with NDA variables and additional sets of constraints. This study includes a review of known formulations, as well as new theoretical results comparing the NDA formulations and constraints (CI≥) presented in the previous section.

3.2.1. The NDA-BASE formulation

Consider the set of binary variables $h_i^d, \forall i,d \in V$, which indicate whether node $d \in V$ plays the role of the depot of the cycle node i is in. Consider, also, the following MILP formulation, denoted by **NDA-BASE**:

$$\text{Min.} \sum_{\{i,j\} \in E} d_{ij} u_{ij} \qquad \text{(OBJ)}$$

s.t. (DEG), (pDEP), (BIN-u), (BIN-y),

$$\sum_{d \in V} h_i^d = 1, \qquad \forall i \in V \qquad \text{(BAS-2)}$$

$$h_i^d \leq h_d^d, \qquad \forall i,d \in V \qquad \text{(BAS-3)}$$

$$h_i^i = y_i, \qquad \forall i \in V \qquad \text{(BAS-4)}$$

$$h_i^d = 0, \qquad \forall i,d \in V : d > i \qquad \text{(SB)}$$

$$h_i^d \in \{0,1\}, \qquad \forall i,d \in V. \qquad \text{(BIN-}h\text{)}$$

Constraints (BAS-2) indicate that any node i must be assigned to exactly one depot and (BAS-3) indicate that if node d is the depot of the cycle node i is in, then, node d must be a depot. Constraints (BAS-4) relate the h_i^i and y_i variables, stating that a node i is a depot if and only if it is the depot of the cycle it is in. Constraints (SB) state that a node d cannot be the depot of a cycle which also includes some other node $i < d$, therefore enforcing the SB strategy mentioned earlier. Although (SB) are not needed for the formulation to be valid, these constraints lead to considerable improvements to LP relaxation bounds and CPU times. Finally, constraints (BIN-h) define the h_i^d variables as binary.

3.2.2. NDA - preventing solutions with less than p cycles

Consider the following set of constraints, first introduced by Gollowitzer et al. (2014):

$$h_i^d + u_{ij} \leq h_j^d + 1, \qquad \forall i,j \in V : i \neq j, \forall d \in V. \qquad \text{(NDA≥)}$$

These constraints state that if two nodes i and j are adjacent, the depots of the cycles nodes i and j are in must be the same. It is well-known that (NDA≥), together with the constraints in NDA-BASE, define a valid formulation for the HpMP$_{\geq}$, which we denote by **NDA$_{\geq}$**. However, as shown by Erdoğan et al. (2016), these constraints can be generalized to the following set of constraints (which was mentioned in the introduction):

$$\sum_{d \in D} h_i^d + u_{ij} \leq \sum_{d \in D} h_j^d + 1, \qquad \forall i,j \in V : i < j, \forall D \subseteq V. \qquad \text{(GNDA≥)}$$

Their interpretation is analogous to that of constraints (NDA≥). However, unlike (NDA≥), the more general constraints are redundant for $j < i$, since, due to constraints (BAS-2), any such constraint for two nodes $i, j : i > j$ and some set $D \subseteq V$ would be equivalent to constraint (GNDA≥) for the same pair of nodes and for the set $V \setminus D$. This is why we have defined (GNDA≥) only for $i < j$. We denote by $\mathbf{GNDA}_{\geq}$ the formulation resulting from augmenting NDA-BASE with constraints (GNDA≥).

In contrast to constraints (NDA≥), the number of constraints (GNDA≥) grows exponentially as a function of the number of nodes in the graph. However, they can be separated in polynomial time as shown by Erdoğan et al. (2016). Preliminary computational testing has led to some conclusions on their strength, in terms of the quality of LP relaxation bounds. In fact, adding constraints (GNDA≥) to the $\text{NDA}_{\geq}$ formulation appears to lead to considerable improvements to LP relaxation bounds (although those bounds are still not strong). However, if other valid inequalities (as the ones discussed in Sections 3.2.3 and 4) are also included, the improvements resulting from adding (GNDA≥) become less relevant.

The following result relates (GNDA≥) and the cycle constraints (CI≥) introduced in Section 3.1:

Proposition 1. *The $\text{GNDA}_{\geq L}$ formulation implies constraints (CI≥).*

The proof of this result will be given later indirectly. This, together with the results of our preliminary computational test of constraints (GNDA≥), gives some indication of the weakness of the cycle-based inequalities for preventing solutions with fewer than p cycles which have been mentioned in Section 3.1.

3.2.3. NDA - valid inequalities

With the constraints described in the previous section, we obtain valid formulations for the HpMP$_{\geq}$. Valid formulations for the HpMP can be obtained by adding the clique constraints (CC≤) introduced in Section 3.1 to these formulations. However, the LP relaxations of these formulations can be further enhanced with the following valid inequalities, proposed by Erdoğan et al. (2016) and Barbato et al. (2023) respectively:

$$2h_d^d \leq \sum_{i \in V : i > d} h_i^d, \qquad \forall d \in V. \tag{VI-1}$$

$$u_{ij} + \sum_{d=i+1}^{j} h_j^d \leq 1, \qquad \forall i, j \in V : i < j. \tag{VI-2}$$

Constraints (VI-1) state that if some node d is a depot, two other nodes with indices greater than d must be assigned to it, and constraints (VI-2) state that if two nodes i and j are adjacent and $i < j$, then the depot of the cycle node j is in, cannot be any node with an index greater than i. The validity of both sets of constraints is a consequence of the SB strategy, and both sets of valid inequalities have been shown to result in improvements to LP relaxation bounds and computational times (see Erdoğan et al., 2016, Barbato et al., 2023). As observed by Barbato et al. (2023), these improvements are also noticeable when augmenting NDA-BASE with constraints (NDA≥) and (VI-2) as opposed to only augmenting it with (NDA≥). However, if NDA-BASE is augmented with constraints (GNDA≥) instead of (NDA≥), constraints (VI-2) become redundant, as they are a subset of (GNDA≥). Indeed, it can be checked that constraint (GNDA≥) for nodes $i, j : i < j$ and for $D = \{1, \ldots, i\}$ is equivalent to constraint (VI-2) for the same pair of nodes when constraints (SB) are also considered. On a related note, we also point out that a (new) generalization of constraints (VI-1) is presented in Section 4.

4. Augmenting NDA and introducing the EDA formulations

In this section, we consider new formulations that produce substantially better LP relaxation bounds than the formulations previously described. These formulations include EDA variables (binary variables $z_{ij}^d, \forall \{i, j\} \in E, \forall d \in V$ indicating if node d is the depot of the cycle edge $\{i, j\}$ is in) in contrast to the previously discussed formulations that only use NDA variables (in addition to the u_{ij} and y_i variables). We also relate the LP relaxations of the new formulations to the LP relaxations of the previous formulations. In particular, we present new sets of constraints which include the edge and NDA variables that are implied by constraints in the EDA formulations.

Consider the following constraints:

$$\sum_{d \in V} z_{ij}^d = u_{ij}, \qquad \forall \{i, j\} \in E \tag{AUG-1}$$

$$Z^d(E(i, V \setminus \{i\})) = 2h_i^d, \quad \forall i, d \in V : d \leq i \tag{AUG-2}$$

$$z_{ij}^d \leq h_i^d, \qquad \forall \{i, j\} \in E : i < j, \forall d \in V \tag{AUG≥-1}$$

$$z_{ij}^d \leq h_j^d, \qquad \forall \{i, j\} \in E : i < j, \forall d \in V \tag{AUG≥-2}$$

$$z_{ij}^d = 0, \qquad \forall \{i, j\} \in E, \forall d \in V : d > \min\{i, j\} \tag{AUG-SB}$$

$$z_{ij}^d \in \{0, 1\}, \qquad \forall \{i, j\} \in E, \forall d \in V. \tag{BIN-z}$$

Constraints (AUG-1) link the z_{ij}^d and u_{ij} variables (by stating that an edge is used if and only if it is assigned to exactly one depot). Constraints (AUG-2) link the EDA and NDA variables (by stating that a node is assigned to a depot if and only if both edges incident on it are also assigned to that depot) and constraints (AUG≥-1) and (AUG≥-2) state that if some edge $\{i, j\}$ is assigned to some depot d, then, both nodes i and j must be assigned to the same depot d. These constraints are similar to constraints appearing in formulations for facility location problems, in particular, for defining the so-called "strong" location formulations (see Wolsey, 2020) and, similarly to the impact that their inclusion has in formulations for the location problem and variants, they also have a strong impact in the formulations developed in this paper. Finally, constraints (AUG-SB) enforce the SB strategy and (BIN-z) define the new variables as binary. We denote by $\mathbf{AugNDA}_{\geq}$ the formulation resulting from augmenting NDA-BASE with these sets of constraints. We will later present results which indirectly imply the validity of $\text{AugNDA}_{\geq}$ for the HpMP$_{\geq}$, and thus we do not directly prove the validity of this formulation.

To give some intuition of what can be obtained by adding the EDA variables to the formulations of Section 3 and to indirectly show the strength of the EDA formulations, we begin by considering a subformulation of $\text{AugNDA}_{\geq}$ obtained by removing constraints (AUG-2) from $\text{AugNDA}_{\geq}$. We denote it by $\mathbf{AugNDA}_{\geq}^{-}$.

The next proposition addresses an observation mentioned in the introduction, namely, by establishing a strong relation between the sets of feasible solutions of the $\text{AugNDA}_{\geq L}^{-}$ and the $\text{GNDA}_{\geq L}$ formulations. To facilitate the statement of the proposition, we write two subformulations, A and B, of $\text{AugNDA}_{\geq L}^{-}$ and $\text{GNDA}_{\geq L}$ respectively. A is given by:

$$\sum_{d \in V} h_i^d = 1, \qquad \forall i \in V \tag{BAS-2}$$

$$\sum_{d \in V} z_{ij}^d = u_{ij}, \qquad \forall \{i, j\} \in E \tag{AUG-1}$$

$$z_{ij}^d \leq h_i^d, \qquad \forall \{i, j\} \in E : i < j, \forall d \in V \tag{AUG≥-1}$$

$$z_{ij}^d \leq h_j^d, \qquad \forall \{i, j\} \in E : i < j, \forall d \in V \tag{AUG≥-2}$$

$$u_{ij} \geq 0, \qquad \forall \{i, j\} \in E \tag{$u \geq$}$$

$$h_i^d \geq 0, \qquad \forall i, d \in V \tag{$h \geq$}$$

$$z_{ij}^d \geq 0, \qquad \forall \{i, j\} \in E, \forall d \in V. \tag{$z \geq$}$$

And B is given by:

$$\sum_{d \in V} h_i^d = 1, \qquad \forall i \in V \tag{BAS-2}$$

$$\sum_{d \in D} h_i^d + u_{ij} \leq \sum_{d \in D} h_j^d + 1, \qquad \forall \{i, j\} \in E : i < j, \forall D \subseteq V \tag{GNDA≥}$$

$$u_{ij} \geq 0, \qquad \forall \{i, j\} \in E \tag{$u \geq$}$$

$$h_i^d \geq 0, \qquad \forall i, d \in V. \tag{$h \geq$}$$

Then, the following result holds:

Proposition 2. *$ADM(B) = PROJ_{u,h}(ADM(A))$.*

The proof of this result is given in the supplementary material. An immediate consequence of Proposition 2 is the following result:

Proposition 3. *$ADM(GNDA_{\geq L}) = PROJ_{u,h}(ADM(AugNDA^{-}_{\geq L}))$.*

This result implies the validity of the $\text{AugNDA}^{-}_{\geq}$ formulation and also states that, in terms of LP relaxations, adding the exponentially sized set of constraints (GNDA≥) to the NDA-BASE_L formulation corresponds to adding the z^d_{ij} variables and the polynomially sized sets of constraints (AUG-1), (AUG≥-1), (AUG≥-2) and ($z \geq$) to the same formulation.

We relate, next, the $\text{AugNDA}^{-}_{\geq L}$ formulation with the cycle inequalities (CI≥), indirectly proving Proposition 1 (the proof of which had not been given before):

Proposition 4. *The $\text{AugNDA}^{-}_{\geq L}$ formulation implies constraints (CI≥).*

The proof of this result is given in the supplementary material. Two comments are in order. First, this result also demonstrates the validity of constraints (CI≥). Moreover, the proof of this result produces the following stronger set of path constraints:

$$U(P) + Y(P \setminus \{\min(P)\}) \leq |P|, \qquad \forall \text{ path } P. \qquad \text{(PI≥)}$$

Observe that, given a cycle C with nodes $\{i_1, \ldots, i_{|C|}\}$ and a path P with the same nodes (in the same order), adding $u_{i_{|C|}i_1} \leq 1$ to the path inequality for path P leads to the cycle inequality (CI≥) for cycle C.

Computational tests reveal substantial differences in the quality of the LP relaxation bounds produced by the $\text{AugNDA}_{\geq}$ and $\text{AugNDA}^{-}_{\geq}$ formulations. This suggests that there exist interesting valid inequalities (other than constraints (GNDA≥)) defined only on the space of the u_{ij} and h^d_i variables which are not redundant in the LP relaxation of the $\text{AugNDA}^{-}_{\geq}$ formulation, and which might lead to considerable improvements to its LP relaxation bounds.

We now describe new valid inequalities that differ from constraints (GNDA≥) in the sense that constraints (GNDA≥) are defined for edges (that is, we have one constraint (GNDA≥) for each edge of the graph and for each subset of nodes $D \subseteq V$), while the new constraints are defined for subsets of edges. We begin by first considering the simplest (though very relevant) set of valid inequalities:

$$2h^d_i + U(E(i,S)) \leq \sum_{j \in S} h^d_j + 2,$$

$$\forall i, d \in V : d \leq i, \forall S \subseteq V \setminus \{i\} : |S| \geq 3. \qquad \text{(VI-3)}$$

These constraints can be interpreted as follows: if node i is assigned to depot d and is adjacent to some nodes in S, then some of these nodes must be assigned to depot d. Observe that (VI-3) for a given i, d and $S : |S| \geq 3$, is stronger than the inequality obtained by adding (NDA≥) for i, d and every node $j \in S$.

Unlike other sets of constraints discussed afterwards, (VI-3) can be separated in polynomial time using a procedure (see the supplementary material) similar to the one proposed by Erdoğan et al. (2016) to separate constraints (GNDA≥). However, since the separation algorithms for (GNDA≥) and (VI-3) are exact and the LP relaxation bounds obtained with these constraints still do not coincide with those obtained with $\text{AugNDA}_{\geq}$, we conclude that there exist other valid inequalities that are necessary to describe $\text{PROJ}_{u,h}(\text{ADM}(\text{AugNDA}_{\geq L}))$. One such set can be obtained from (VI-3) by generalizing for subsets of potential depots, similarly to how (NDA≥) can be generalized to (GNDA≥):

$$\sum_{d \in D} 2h^d_i + U(E(i,S)) \leq \sum_{d \in D} \sum_{j \in S} h^d_j + 2,$$

$$\forall i \in V, \forall D \subseteq V, \forall S \subseteq V \setminus \{i\} : |S| \geq 3. \qquad \text{(VI-4)}$$

The interpretation of these constraints remains mostly unchanged from the interpretation provided for constraints (VI-3), with the only difference being that single depots are now replaced with sets of depots. These constraints can also be separated in polynomial time (see the supplementary material), although the separation algorithm is more complex than the one used to separate constraints (VI-3).

These valid inequalities can be further generalized into:

$$\sum_{d \in D_1} 2h^d_i + \sum_{d \in D_2} h^d_i + U(E(i,S)) \leq \sum_{d \in D_1} \sum_{j \in S} h^d_j + \sum_{d \in D_2} \sum_{j \in S \setminus \{f(d)\}} h^d_j + 2, \quad \forall i \in V,$$

$$\forall D_1, D_2 \subseteq V : D_1 \cap D_2 = \emptyset, \forall S \subseteq V \setminus \{i\} : |S| \geq 3, \forall f : D_2 \to S. \qquad \text{(VI-5)}$$

Where f is a function that, given a node $d \in D_2$, returns a node $j \in S$. We denote by $\textbf{GGNDA}_{\geq}$ the formulation resulting from augmenting $\text{GNDA}_{\geq}$ with constraints (VI-5).

The interpretation of these constraints is not as intuitive as that of constraints (VI-4). Assume that node i is adjacent to two nodes in S. If i is assigned to $d_1 \in D_1$, then at least two nodes in S must also be assigned to d_1. If i is assigned to $d_2 \in D_2$, then, of all nodes in S other than node $j = f(d_2)$, at least one must be assigned to d_2. This is because if i is adjacent to two nodes in S and we pick a node j assigned to d_2, at least one other node in S must also be assigned to d_2. If j is not assigned to d_2, then two other nodes in S must still be assigned to d_2.

As mentioned earlier, the valid inequalities (VI-5) generalize (VI-4). This can be seen by considering $D_2 = \emptyset$ in (VI-5). To provide some insight as to when some constraints (VI-5) are violated when all (VI-4) are satisfied, consider feasible solution $(\mathbf{u}^*, \mathbf{h}^*)$ for NDA-BASE_L satisfying (VI-4), and consider one constraint (VI-4) for node i and two sets S, D. Observe that a new inequality (VI-5) can be obtained from the previous one by considering D_2 as a subset of D and by subtracting $\sum_{d \in D_2} h^d_i$ from the left-hand side and $\sum_{d \in D_2} h^d_{f(d)}$ from the right-hand side. If $\sum_{d \in D_2} \mathbf{h}^{d*}_{f(d)} > \sum_{d \in D_2} \mathbf{h}^{d*}_i$ holds for this solution, then the new constraint (VI-5) we just obtained could be violated, even though all constraints (VI-4) were satisfied.

Unlike what happened with the previous two sets of valid inequalities, it is not clear whether there exists a polynomial-time algorithm to separate the more general constraints (VI-5) for fractional solutions. However, a heuristic separation algorithm that appears to work well in practice determines the most violated inequality (VI-5) for each node i by first running the separation algorithm discussed in the supplementary material for constraints (VI-4) and then iteratively modifying sets S, D_1 and D_2 in order to maximize the violation of the resulting inequality. This algorithm is discussed in detail in the supplementary material.

The results of our computational tests indicate that, with constraints (VI-3), we obtain a formulation that, for most of the benchmark instances, produces LP relaxation bounds close to those produced by $\text{AugNDA}_{\geq}$. However, this is not the case for all instances. In particular, this difference becomes far more substantial for instances whose edge weight functions do not satisfy the triangle inequality. We will make this more precise in Section 6.1. The new set of inequalities (VI-5), however, leads to new insights on the development of efficient methods to solve such instances, since this difference in LP relaxation bounds is no longer considerable if these constraints are included. Before addressing whether these inequalities are sufficient to describe $\text{PROJ}_{u,h}(\text{ADM}(\text{AugNDA}_{\geq L}))$, we show they are implied by $\text{AugNDA}_{\geq L}$.

Proposition 5. *$PROJ_{u,h}(ADM(AugNDA_{\geq L})) \subseteq ADM(GGNDA_{\geq L})$.*

Proof. We prove that constraints (VI-5) are implied by some constraints in the $\text{AugNDA}_{\geq L}$ formulation. To do so, we begin by adding constraints (AUG-1) for every edge $\{i,j\} \in E : j \in S$. This leads to:

$$U(E(i,S)) = \sum_{j \in S} \sum_{d \in V} z^d_{ij} = \sum_{d \in V} \sum_{j \in S} z^d_{ij}$$

This equality can then be rewritten as follows:

$$U(E(i,S)) = \sum_{d \in V \setminus (D_1 \cup D_2)} \sum_{j \in S} z^d_{ij} + \sum_{d \in D_1} \sum_{j \in S} z^d_{ij} + \sum_{d \in D_2} \sum_{j \in S} z^d_{ij}$$

The right-hand side of this equality has been split into three summation terms, and we bound each term from above differently. For every $d \in V \setminus (D_1 \cup D_2)$, we consider $\sum_{j \in S} z^d_{ij} \leq 2h^d_i$ (by using constraints

Table 1
Summarizing EDA and NDA formulation relations.

$\mathbf{u_{ij}}$, $\mathbf{h_i^d}$, $\mathbf{z_{ij}^d}$		$\mathbf{u_{ij}}$, $\mathbf{h_i^d}$
$\sum_{d\in V} z_{ij}^d = u_{ij}$ (i.e., (AUG-1)) $z_{ij}^d \le h_i^d$, $z_{ij}^d \le h_j^d$ (i.e., (AUG≥-1), (AUG≥-2))	↔	• $\sum_{d\in D} h_i^d + u_{ij} \le \sum_{d\in D} h_j^d + 1$
(AUG-1), (AUG≥-1), (AUG≥-2) $Z^d(E(i, V\setminus\{i\})) = 2h_i^d$ (i.e., (AUG-2))	→	• $\sum_{d\in D} h_i^d + u_{ij} \le \sum_{d\in D} h_j^d + 1$ • $\sum_{d\in D_1} 2h_i^d + \sum_{d\in D_2} h_i^d + U(E(i,S)) \le$ $\le \sum_{d\in D_1}\sum_{j\in S} h_j^d + \sum_{d\in D_2}\sum_{j\in S\setminus\{f(d)\}} h_j^d + 2$ • $\sum_{i\in A}\sum_{d\in D} 2h_i^d + U(E(A,S)) \le \sum_{j\in S}\sum_{d\in D} 2h_j^d + 2\lvert A\rvert$

(AUG-2)). For every $d \in D_1$, we consider $\sum_{j\in S} z_{ij}^d \le \sum_{j\in S} h_j^d$ (by using constraints (AUG≥-1) and (AUG≥-2) to state that $z_{ij}^d \le h_j^d$, $\forall j \in S$). Finally, for every $d \in D_2$, we consider $\sum_{j\in S} z_{ij}^d \le h_i^d + \sum_{j\in S\setminus\{f(d)\}} h_j^d$ (by using constraints (AUG≥-1) and (AUG≥-2) to state $z_{if(d)}^d \le h_i^d$ and $z_{ij}^d \le h_j^d$, $\forall j \in S\setminus\{f(d)\}$). This leads to the following inequality:

$$U(E(i,S)) \le \sum_{d\in V\setminus(D_1\cup D_2)} 2h_i^d + \sum_{d\in D_1}\sum_{j\in S} h_j^d + \sum_{d\in D_2}\left(h_i^d + \sum_{j\in S\setminus\{f(d)\}} h_j^d\right)$$

Finally, by using equality (BAS-2) for node i to rewrite $\sum_{d\in V\setminus(D_1\cup D_2)} 2h_i^d$ as $2 - \sum_{d\in(D_1\cup D_2)} 2h_i^d$, we obtain:

$$\sum_{d\in D_1} 2h_i^d + \sum_{d\in D_2} h_i^d + U(E(i,S)) \le \sum_{d\in D_1}\sum_{j\in S} h_j^d + \sum_{d\in D_2}\sum_{j\in S\setminus\{f(d)\}} h_j^d + 2.$$

□

We conclude this section by returning to the description of $\text{PROJ}_{u,h}(\text{ADM}(\text{AugNDA}_{\ge L}))$. For very small instances, we have noticed that the LP relaxation bound given by $\text{AugNDA}_{\ge}$ is still better than the LP relaxation bound given by $\text{GGNDA}_{\ge}$ (constraints (VI-5) were separated by enumeration). In fact, constraints (VI-4) can be generalized in a different way to how they were generalized earlier, more precisely, by generalizing a cut-set $[\{i\}, S]$ to a cut-set $[A, S]$ resulting in the following set of valid inequalities:

$$\sum_{i\in A}\sum_{d\in D} 2h_i^d + U(E(A,S)) \le \sum_{j\in S}\sum_{d\in D} 2h_j^d + 2|A|,$$
$$\forall A, S, D \subseteq V : |A|, |S| \ge 3. \qquad \text{(VI-6)}$$

These constraints state that, if there are many nodes in A assigned to depot d and many edges linking nodes in A and S, then there must be some nodes in S assigned to d. Constraints (VI-6) can also be shown to be implied by $\text{AugNDA}_{\ge L}$, but for brevity, we omit the derivation proof in this paper (it is similar to the proof for constraints (VI-5)). This generalization is presented primarily from a theoretical perspective as our experiments have shown that it does not play a significant role in the solution methods. Finally, and still from a theoretical point of view, in the same way that these inequalities generalize (VI-4), it would be interesting to find a generalization of (VI-5), implied by $\text{AugNDA}_{\ge L}$, that also generalizes (VI-6).

Table 1 summarizes the relations between the EDA and NDA formulations mentioned above, highlighting the sets of constraints that differ between the formulations. Observe that we have a proven equivalence on the first row and (only) a proven implication on the second row.

We conclude this section by observing that a formulation using only the z_{ij}^d variables can be obtained from the formulation $\text{AugNDA}_{\ge}$. A straightforward variable substitution using the linking equalities (AUG-1) and (AUG-2) allows us to eliminate the h_i^d and u_{ij} variables from $\text{AugNDA}_{\ge}$. For reasons of space, this formulation (as well as a computational comparison with the equivalent formulation $\text{AugNDA}_{\ge}$) is described in the supplementary material.

5. On the generalizability of the SB strategy

All the formulations and theoretical results presented here remain valid without considering the SB strategy (observe that SB is imposed by setting to zero the value of some of the variables of the formulations). In addition, the projection results stated here are proven without considering SB constraints. However, it is also clear that the use of the SB strategy has a strong impact on the efficiency of the proposed methods. Thus, a question worth considering concerns the applicability of this or related strategies to variants of this problem.

In this section, we briefly address two variants of the H*p*MP: one illustrates a situation in which the proposed SB strategy is still applicable (although not in a straightforward way) and the other illustrates a situation where it is not (or at least, it is not clear how to include an SB strategy unless the costs of the variables in the objective function are structured in some very specific way). The first variant includes node setup costs associated with the nodes selected as depots. In this case, the objective function becomes

$$\text{Min.} \sum_{\{i,j\}\in E} d_{ij}u_{ij} + \sum_{i\in V} c_i y_i$$

where c_i is the cost associated with designating node i as a depot. This situation is interesting because, at first glance, the inclusion of these costs invalidates the SB strategy, since selecting the node with the lowest index in a cycle to be the depot can lead to suboptimal solutions. However, sorting the nodes by non-descending order of setup costs before solving the problem allows us to use the SB strategy. Furthermore, the inclusion of node setup costs does not invalidate the theoretical results stated in this paper.

The second situation arises when considering different costs for different node-depot assignments, with or without the setup costs mentioned earlier. In this case, we think that no SB strategy is valid unless the costs associated with the NDA variables are structured in a very specific way (e.g., if there exists some sorting of the nodes such that, for any node i and any two nodes d_1, d_2 such that $d_1 < d_2$, $c_i^{d_1} \le c_i^{d_2}$ holds).

6. Computational experiment

In this section, we present the numerical results from our computational experiment, in which we test valid formulations and algorithms for the H*p*MP and the $\text{H}p\text{MP}_{\ge}$ discussed throughout this paper. We begin by introducing the benchmark instances and hardware and software configurations used in this experiment and follow by presenting and analyzing the aforementioned results. For brevity reasons, in this section, we only present some of the data - for the tables with the full results, see the supplementary material.

6.1. Hardware and software configurations and test instances

The instances considered for this computational experiment are based on graphs from a subset of the set of symmetric instances for the TSP from TSPLIB (Reinelt, 1991), which includes 34 instances - bayg29, swiss42, **att48**, gr48, hk48, **eil51**, **berlin52**, brazil58, **st70**, **eil76**, **pr76**, **rat99**, **kroA100**, **kroB100**, **kroC100**, **kroD100**, **kroE100**, **rd100**, **lin105**, **pr124**, **bier127**, **pr136**, **ch150**, **kroB150**, **u159**, brg180, **rat195**, **d198**, **tsp225**, **pr226**, **gil262**, **a280**, **lin318**, **rd400** - with node counts ranging from 29 to 400. Following the literature on the H*p*MP (see Erdoğan et al., 2016, Bektaş et al., 2019, Barbato et al., 2023, Barbato & Gouveia, 2024), the instances whose names are in bold have

their edge weight functions modified to the Euclidean distance without rounding to integer values.

The instances based on the brg180 graph were only recently incorporated into the benchmark set (see Barbato & Gouveia, 2024). The computational results reported in that study highlighted these as particularly challenging, especially compared with other instances, when evaluated using their proposed method. Most methods proposed in the current paper reflect a similar behavior, notably in the optimal value of the LP relaxations, and this difficulty also carries over to the proposed branch-and-cut approach.

Based on these observations, we performed a more detailed analysis of the brg180 graph. In particular, most instances in the benchmark set, with the exception of brg180, either satisfy the triangle inequality or contain only a small proportion of triples (i, j, k) such that $d_{ik} + d_{kj} < d_{ij}$, (up to 4.16 % for brazil58, below 2 % for other instances relative to the total number of possible triples such that $i < j$ and $k \neq i, j$). In contrast, the brg180 instance exhibits a significantly higher proportion (approximately 16.57 %) of such triples. Observe that this percentage is not as small as it appears since, for any graph with edge weights, this proportion can never exceed 1/3 of the total number of triples. To see this, consider any triple (i, j, k). Whenever we have $d_{ik} + d_{kj} < d_{ij}$ then we also have $d_{ij} + d_{jk} > d_{ik}$ and $d_{ji} + d_{ik} > d_{jk}$, and hence, for every triple for which the corresponding triangle inequality is violated, there exist two other triples for which that is not the case. Observe also that it is not difficult to find edge weights for which the upper limit of 1/3 is obtainable, indicating that the bound of 1/3 is tight.

To further investigate this phenomenon, we generated new instances based on three "similar" graphs with 50, 75, and 100 nodes, respectively. The edge weights for these instances were randomly generated following a discrete uniform distribution in the range $[1, 100]$ and exhibit a comparable structure, with the proportion of triangle inequality violations measured at 15.46 %, 16.29 % and 16.02 % for the three underlying graphs, respectively. Due to the apparent additional difficulty in solving these instances, we refer to the randomly generated and brg180-based instances as "Difficult" (D) instances and to all other instances as "Standard" (S) instances.

For each of the graphs, eight different values of p are considered (see Table 2):

Table 2
Values of p considered for instances for the HpMP.

p_1	p_2	p_3	p_4	p_5	p_6	p_7	p_8
$\lfloor n/30 \rfloor$	$\lfloor n/15 \rfloor$	$\lfloor n/10 \rfloor$	$\lfloor n/7 \rfloor$	$\lfloor n/5 \rfloor$	$\lfloor n/4 \rfloor$	$\lceil (p_6 + p_8)/2 \rceil$	$\lfloor n/3 \rfloor$

We also only consider values of p that are greater than or equal to 2 (for example, we only consider instances based on the bayg29 graph with $p \geq p_3$, since $p_2 = 1$ and $p_1 = 0$). When testing for the HpMP$_{\geq}$, only p_5 to p_8 are considered since the constraints that prevent solutions with fewer than p cycles become more relevant for values of p closer to $n/3$, as mentioned earlier. Note that, for some of these graphs, the tests for instances with $p \in \{p_1, p_2\}$ are not reported on in other literature.

We present results on the CPU times for solving the instances of the HpMP and HpMP$_{\geq}$ as well as *i*) the LP relaxation bounds and *ii*) the Root Node (RN) relaxation bounds. We follow Barbato et al. (2023) and consider the optimal value of the RN relaxation to be the lower bound provided by the solver (CPLEX) after solving the LP relaxation at the root node of the branch-and-bound tree and generating general-purpose cuts and applying other forms of preprocessing to improve the initial lower bounds. These two lower bounds are evaluated through the gaps calculated as $(UB - LB)/UB \times 100\,\%$, where UB is the best upper bound known to date (often the optimal value), and LB is the lower bound in question.

All tests were run on an HP Z1 G9 PC with an Intel® Core™ i7-13700 CPU, 32GB of DDR5-4400 RAM running Ubuntu 22.04, and all programs were compiled with g++, version 11.4.0. The formulations were implemented using Concert Technology for C++ and the test instances were solved using CPLEX 22.1.1. All CPLEX parameters, other than those specified in Section 6.4, were set to their default values.

6.2. Algorithm definitions

In this experiment, several different branch-and-cut algorithms based on formulations discussed in the previous sections are tested. We begin by listing the algorithms compared for the HpMP$_{\geq}$:

- $\mathcal{AUG}^{+}_{\geq}$ - Results from implementing the AugNDA$_{\geq}$ formulation for use with CPLEX;
- $\mathcal{NDA}_{\geq}$ - Branch-and-cut algorithm based on the NDA$_{\geq}$ formulation augmented with constraints (VI-1) and (VI-2);
- $\mathcal{NDA}^{+}_{\geq}$ - Branch-and-cut algorithm based on the NDA$_{\geq}$ formulation augmented with constraints (VI-1), (VI-2) and (VI-3).

Since these are algorithms for the HpMP$_{\geq}$, the only other algorithms / formulations known from the literature with which these can be compared are those proposed by Barbato et al. (2023). In particular, we compare $\mathcal{AUG}^{+}_{\geq}$ and $\mathcal{NDA}^{+}_{\geq}$ with an algorithm based in one of the aggregated NDA formulations mentioned in the introduction:

- $\mathcal{SANDA}^{+}_{\geq}$ - Results from implementing the SANDA^{+} formulation for use with CPLEX.

The SANDA^{+} formulation (see Barbato et al., 2023, now denoted by SANDA$^{+}_{\geq}$) was chosen because it is the formulation with the most promising computational results studied by Barbato et al. (2023). We note that SANDA$^{+}_{\geq}$ is a compact formulation that includes the variables u_{ij} and y_i, in addition to integer variables h_i indicating the index of the depot of the cycle node i is in, in contrast to the disaggregated variables h^d_i appearing in the NDA formulations studied in this paper.

Based on the comparison of algorithms for the HpMP$_{\geq}$, one algorithm for the HpMP was also tested:

- $\mathcal{NDA}^{+}$ - Branch-and-cut algorithm based on the NDA$_{\geq}$ formulation augmented with constraints (VI-1), (VI-2), (VI-3) and (CC≤).

For more information on the separation algorithms used to separate the exponentially sized constraints in these formulations, we forward the reader to the supplementary material. Note that in the $\mathcal{NDA}_{\geq}$, $\mathcal{NDA}^{+}_{\geq}$ and $\mathcal{NDA}^{+}$ algorithms, constraints (VI-1) were included from the start and constraints (BAS-3), (VI-2) and (NDA≥) were added dynamically, instead of being included in the formulation from the start (despite not being exponentially sized). They were, however, separated by enumerating the entire set of constraints, and thus, a separation algorithm for those constraints is not described here.

6.3. Primal heuristic

We use the hybrid genetic algorithm (HGA) proposed by He et al. (2024). This algorithm is executed only once and for the duration of $\lfloor n/20 \rfloor$s before solving any instance of the HpMP or HpMP$_{\geq}$, where n is the number of nodes in the instance's graph. The solution found by this algorithm is then provided to the exact algorithm using Concert Technology's `addMIPStart` function. This is done for every algorithm proposed in the current work. Note that the time spent by the HGA is also included in the computational times reported for the algorithms.

6.4. CPLEX parameters and other implementation details

In every computational test described here, some of CPLEX's parameters are set to non-default values:

- *Time limit:* For all tests (other than when solving LP and RN relaxation bounds), the time limit is set to 1h (in other words, the `IloCplex::Param::TimeLimit` parameter was set to 3600) - if the

time limit is reached in any of these tests, an interval to which the optimal value is guaranteed to belong is reported instead of the optimal value;
- *LP algorithm:* For all tests, only the Dual Simplex algorithm is used to solve the relaxations at any node of the branch-and-bound tree (the values of the `IloCplex::Param::RootAlgorithm` and `IloCplex::Param::NodeAlgorithm` parameters are set to 2);
- *Branching strategy:* For all tests done with algorithms based on formulations with exponentially sized sets of constraints, CPLEX's strong branching is enabled (in other words, the parameter `IloCplex::Param::MIP::Strategy::VariableSelect` is set to 3).

Any parameter not mentioned above was set to its default value, unless stated otherwise. Note that CPLEX can utilize multiple CPU cores when working with compact formulations but can only utilize a single core when working with branch-and-cut algorithms based on formulations with exponentially sized sets of constraints, due to the usage of CPLEX callbacks.

Moreover, in any branch-and-cut algorithm based on a formulation with exponentially sized sets of constraints, separated constraints are added *locally*, that is, they are included only in subproblems in the subtree rooted at the node where the constraint was separated.

In addition, constraints that are considered for inclusion (i.e., added only when they are found to be violated) are separated at every node of the branch-and-bound tree. In our implementation, we have made the choice of adding a constraint if the node of the branch-and-bound tree on which the constraint is found to be violated has a depth of at most 5. If it has depth greater than 5, then a constraint is added only if its slack variable has a value below -0.1. Furthermore, if at any node of the tree, five constraint separation and reoptimization runs do not lead to lower bounds improving by more than 0.01 %, the algorithm proceeds to branching (however, constraints can be separated once again at other nodes of the tree or even at the child nodes of the current node).

6.5. Measuring the impact of valid inequalities (VI-3) and (VI-5)

We begin by comparing the lower bounds provided by $\mathcal{NDA}_{\geq}$ and $\mathcal{NDA}^{+}_{\geq}$. With this comparison, we want to evaluate the effectiveness of constraints (VI-3) in improving the initial lower bounds. Table 3 includes the average LP and RN gaps obtained for all instances, for each value of p, with $\mathcal{NDA}_{\geq}$ and $\mathcal{NDA}^{+}_{\geq}$:

Table 3
Average LP / RN gaps for $\mathcal{NDA}_{\geq}$ and $\mathcal{NDA}^{+}_{\geq}$.

p	S instances		D instances	
	$\mathcal{NDA}_{\geq}$	$\mathcal{NDA}^{+}_{\geq}$	$\mathcal{NDA}_{\geq}$	$\mathcal{NDA}^{+}_{\geq}$
p_5	1.79 % / 0.90 %	1.10 % / 0.47 %	13.81 % / 13.73 %	8.68 % / 8.83 %
p_6	2.46 % / 1.05 %	1.05 % / 0.38 %	20.41 % / 20.23 %	11.69 % / 11.77 %
p_7	3.56 % / 1.50 %	1.11 % / 0.33 %	30.09 % / 28.94 %	15.10 % / 14.53 %
p_8	8.78 % / 5.88 %	4.57 % / 3.06 %	39.76 % / 36.52 %	14.71 % / 10.03 %

One remark is in order. The RN gaps sometimes slightly exceed the LP gaps, especially in the D instances, due to the strategy mentioned at the end of Section 6.4 to avoid stalling where, if several cut generation and reoptimization runs do not lead to substantial improvements to lower bounds, the algorithm proceeds to branching. This means that, in these instances, the algorithm tends to require several cut generation runs to considerably improve the lower bounds, and sometimes exits the root node of the branch-and-bound tree before adding all the constraints necessary to obtain the LP relaxation bound. In some cases, weak RN relaxation bounds relative to LP relaxation bounds can also be a consequence of the time limit, which was not enforced when solving the LP relaxation.

We also note that preliminary testing shows that the reason why the LP and RN gaps are close is not a consequence of this strategy, since, even after disabling it, the gaps do not change considerably. This shows that, for these instances and for $p \leq p_7$, CPLEX's general purpose cut generation algorithms are incapable of considerably improving lower bounds.

The inclusion of constraints (VI-3) leads to a substantial reduction in LP gaps for all values of p. This improvement is especially pronounced for the D instances, which may be due to their original weaker LP relaxation bounds compared to those of the S instances. For S instances, with the use of CPLEX's general purpose cut separation algorithms, these differences are somewhat narrowed but are still very considerable. It is also interesting to note that even after the use of CPLEX's general purpose cut separation algorithms, $\mathcal{NDA}_{\geq}$ still produces lower bounds which are, on average, worse than those produced by $\mathcal{NDA}^{+}_{\geq}$ without CPLEX's cuts. This difference is particularly pronounced for the D instances.

These results are especially interesting, as they highlight the importance of constraints (VI-3) in obtaining strong initial lower bounds and thus good computational performance. However, they also reveal that while the inclusion of these constraints yields relatively tight LP gaps for the S instances, the gaps remain considerable for the D instances. This observation has motivated our investigation of the more general inequalities (VI-5). Preliminary computational testing reveals that, for the S instances, the improvements obtained with (VI-5) are negligible (average RN gaps improve by less than 0.1 % for any value of p), and hence, these constraints are not helpful in solving these instances. Therefore, in the sequel, we do not consider constraints (VI-5) when solving S instances, only when solving D instances. Table 4 presents the results for the D instances when these inequalities are added to $\mathcal{NDA}^{+}_{\geq}$.

Table 4
Average LP / RN gaps for $\mathcal{NDA}^{+}_{\geq}$ with and without (VI-5) (D instances).

p	$\mathcal{NDA}^{+}_{\geq}$	$\mathcal{NDA}^{+}_{\geq}$ w/ (VI-5)
p_5	8.68 % / 8.83 %	1.93 % / 2.35 %
p_6	11.69 % / 11.77 %	1.84 % / 2.34 %
p_7	15.10 % / 14.53 %	1.71 % / 2.65 %
p_8	14.71 % / 10.03 %	1.89 % / 2.69 %

These results show that, for the D instances, constraints (VI-5) are relevant for obtaining strong lower bounds, with average LP gaps improving from more than 10 % when constraints (VI-3) are included to less than 2 % when constraints (VI-5) are also separated. For this reason, in the sequel, constraints (VI-5) will also be considered when solving D instances.

6.6. Computational results for the HpMP$_{\geq}$

For the purpose of interpreting and comparing the results of this section, we recall that $\mathcal{NDA}^{+}_{\geq}$ is a cutting plane algorithm that includes a separation algorithm for constraints (VI-3) (and for constraints (VI-5) when solving the D instances), and that $\mathcal{AUG}^{+}_{\geq}$ is based on a compact formulation that uses the additional z^{d}_{ij} variables. In theory, the LP relaxation bound of the latter model is at least as good as the LP relaxation bound of the formulation associated with the former method.

We begin by analyzing the results obtained for S instances. The average LP and RN gaps for each algorithm and for each value of p can be found in Table 5, and average CPU times (and the percentage of test runs for which the time limit was reached) are in Table 6:

The average results are separated into instances with up to 199 nodes ("$n \leq 199$") and instances with over 200 nodes ("$n \geq 200$"). This separation was made to allow for a comparison between $\mathcal{NDA}^{+}_{\geq}$, $\mathcal{SANDA}^{+}_{\geq}$ and $\mathcal{AUG}^{+}_{\geq}$, since the latter was only tested for $n \leq 199$ (due to the large number of variables and constraints in AugNDA$_{\geq}$, on which $\mathcal{AUG}^{+}_{\geq}$ is based, which leads to considerable memory requirements for larger instances). From analyzing these results, some conclusions are drawn:

1. *Lower bounds (overview)*: The formulations on which $\mathcal{AUG}^{+}_{\geq}$ and $\mathcal{NDA}^{+}_{\geq}$ are based provide, in general, strong (and similar) LP re-

Table 5
Average LP / RN gaps for $\mathcal{AUG}^{+}_{\geq}$, $\mathcal{NDA}^{+}_{\geq}$ and $\mathcal{SANDA}^{+}_{\geq}$ (S instances).

p	$\mathcal{SANDA}^{+}_{\geq}$ ($n \leq 199$)	$\mathcal{NDA}^{+}_{\geq}$ ($n \leq 199$)	$\mathcal{AUG}^{+}_{\geq}$ ($n \leq 199$)	$\mathcal{SANDA}^{+}_{\geq}$ ($n \geq 200$)	$\mathcal{NDA}^{+}_{\geq}$ ($n \geq 200$)
p_5	2.03 % / 1.21 %	1.04 % / 0.41 %	0.86 % / 0.62 %	2.39 % / 1.55 %	1.40 % / 0.78 %
p_6	3.26 % / 1.88 %	1.02 % / 0.32 %	0.70 % / 0.34 %	3.49 % / 2.47 %	1.21 % / 0.65 %
p_7	5.14 % / 2.99 %	1.12 % / 0.33 %	0.69 % / 0.33 %	5.06 % / 3.58 %	1.10 % / 0.32 %
p_8	11.27 % / 8.27 %	4.10 % / 2.67 %	3.79 % / 2.34 %	13.99 % / 12.16 %	6.69 % / 4.79 %

Table 6
Average CPU times / % of unsolved instances for $\mathcal{AUG}^{+}_{\geq}$, $\mathcal{NDA}^{+}_{\geq}$ and $\mathcal{SANDA}^{+}_{\geq}$ (S instances).

p	$\mathcal{SANDA}^{+}_{\geq}$ ($n \leq 199$)	$\mathcal{NDA}^{+}_{\geq}$ ($n \leq 199$)	$\mathcal{AUG}^{+}_{\geq}$ ($n \leq 199$)	$\mathcal{SANDA}^{+}_{\geq}$ ($n \geq 200$)	$\mathcal{NDA}^{+}_{\geq}$ ($n \geq 200$)
p_5	14.2s / 0.00 %	31.8s / 0.00 %	781.1s / 14.81 %	1230.9s / 33.33 %	2325.7s / 50.00 %
p_6	27.2s / 0.00 %	11.4s / 0.00 %	390.0s / 3.70 %	1814.3s / 50.00 %	1815.1s / 50.00 %
p_7	446.8s / 11.11 %	169.9s / 3.70 %	276.8s / 0.00 %	2431.0s / 66.67 %	1282.6s / 66.67 %
p_8	2212.8s / 55.56 %	700.6s / 14.81 %	405.6s / 7.41 %	3600.0s / 100.00 %	3600.0s / 100.00 %

laxation bounds for S instances, often better than the ones produced by $\mathcal{SANDA}^{+}_{\geq}$. Moreover, even though the LP relaxation bounds provided by the first two approaches are strong, the use of CPLEX's general purpose cut generation algorithms leads to considerably better lower bounds in any of the three tested algorithms;

2. *Lower bounds ($p = p_8$)*: For $\mathcal{AUG}^{+}_{\geq}$ and $\mathcal{NDA}^{+}_{\geq}$ and for most values of p, both the LP and RN gaps are close to or below 1 % on average. However, obtaining similar lower bounds for instances with $p = p_8$ remains challenging. Even with $\mathcal{AUG}^{+}_{\geq}$ (which is the algorithm that provides the strongest lower bounds), RN gaps close to 2 % are obtained. However, using $\mathcal{AUG}^{+}_{\geq}$ is not feasible for instances with $n \geq 200$, given this algorithm's time and memory requirements;
3. *Computational performance (comparing $\mathcal{NDA}^{+}_{\geq}$ and $\mathcal{AUG}^{+}_{\geq}$)*: $\mathcal{NDA}^{+}_{\geq}$ requires substantially less CPU time than $\mathcal{AUG}^{+}_{\geq}$ unless $p = p_8$, in which case, $\mathcal{AUG}^{+}_{\geq}$ performs significantly better. This might be explained by the worse LP relaxation bounds that are produced by the formulation on which $\mathcal{NDA}^{+}_{\geq}$ is based for these instances;
4. *Computational performance ($\mathcal{NDA}^{+}_{\geq}$, $n \leq 199$)*: Every S instance with $n \leq 199$ and $p < p_8$ has been solved to optimality by $\mathcal{NDA}^{+}_{\geq}$, with the exception of the instance based on bier127 with $p = p_7$;
5. *Computational performance ($\mathcal{AUG}^{+}_{\geq}$, $n \leq 199$)*: In contrast to this, $\mathcal{AUG}^{+}_{\geq}$ has not been able to solve several instances with $p = p_5$. Indeed, the results indicate $\mathcal{AUG}^{+}_{\geq}$ is less effective for instances with $p = p_5$ (and, to a lesser extent, $p = p_8$) and is more effective for instances such that $p = p_6$ or $p = p_7$. This is surprising, as it contradicts the finding that instances with greater values of p are more difficult (as reported by Erdoğan et al. (2016), Bektaş et al. (2019), Barbato et al. (2023), Barbato and Gouveia (2024)). We also note that this holds for several graphs (for instance, pr76, rat99, kroD100). This appears to be due to worse computational times in the resolution of the LP relaxations of those instances, which improve as p increases. In fact, in many instances, if $p = p_8$, the LP relaxation is solved in less than 30 % of the time it takes to solve the same LP relaxation with $p = p_5$, and for tested instances with n close to 200, that percentage can be as low as 10 %;
6. *Computational performance ($\mathcal{NDA}^{+}_{\geq}$, $n \geq 200$)*: Worse computational times are obtained with $\mathcal{NDA}^{+}_{\geq}$ for instances with $n \geq 200$. From this set of instances, none with $p = p_8$ have been solved to optimality. For those with lower values of p, around half have been solved to optimality. Nevertheless, in most unsolved instances, optimality gaps are low (in many cases, below 2 %);
7. *Computational performance ($\mathcal{SANDA}^{+}_{\geq}$)*: $\mathcal{SANDA}^{+}_{\geq}$ exhibits good performance for instances with $p \leq p_6$, and for such instances, it is competitive with $\mathcal{NDA}^{+}_{\geq}$. However, computational times deteriorate substantially as p gets closer to $n/3$, with $\mathcal{NDA}^{+}_{\geq}$ performing better than $\mathcal{SANDA}^{+}_{\geq}$ for $p \geq p_7$. These findings are in line with what was reported by Barbato et al. (2023). Furthermore, although it is not visible in Table 6, $\mathcal{SANDA}^{+}_{\geq}$ has, in several instances (27 in total), resulted in the computer running out of memory. These issues appear to be a consequence of the very large number of sub-problems generated by CPLEX, which, in turn, are a consequence of the weak initial lower bounds provided by this approach.

We analyze next the results obtained for the D instances. Table 7 includes the average LP and RN gaps for each algorithm and for each value of p, and Table 8 includes the average CPU times and the percentage of test runs for which the time limit was reached:

As we did previously for the S instances, we now provide a few remarks on the results for the D instances:

1. *Lower bounds (overview)*: In contrast to what we had for the S instances, the lower bounds obtained with $\mathcal{AUG}^{+}_{\geq}$ and $\mathcal{NDA}^{+}_{\geq}$ are very different, the former producing gaps of less than 2 %, while the latter typically produces gaps of more than 10 %. However, the lower bounds obtained with the latter can be improved by including constraints (VI-5). The RN relaxation bounds do not appear to improve substantially over the LP relaxation bounds. Finally, $\mathcal{SANDA}^{+}_{\geq}$ provides the worst lower bounds, with gaps of over 40 % in some cases;
2. *Lower bounds ($p = p_8$)*: Compared to the S instances, the gaps for the D instances are better, both when using $\mathcal{AUG}^{+}_{\geq}$ and $\mathcal{NDA}^{+}_{\geq}$ with (VI-5). This is not the case with $\mathcal{NDA}^{+}_{\geq}$ and $\mathcal{SANDA}^{+}_{\geq}$, which provide much better lower bounds for S instances. Once again, RN gaps are better than LP gaps, but not considerably so;
3. *Computational performance (comparing $\mathcal{NDA}^{+}_{\geq}$ and $\mathcal{AUG}^{+}_{\geq}$)*: Concerning the computational times required by each algorithm, the conclusions drawn for the S instances no longer hold, with $\mathcal{AUG}^{+}_{\geq}$ performing much better than any other algorithm considered in this analyzis, in part due to the differences mentioned above in the lower bounds. Furthermore, an analyzis of CPLEX's logs suggests that this is also due to CPLEX's heuristics that determine solutions whose value is close to the optimal value much earlier with $\mathcal{AUG}^{+}_{\geq}$ than with $\mathcal{NDA}^{+}_{\geq}$. Moreover, $\mathcal{AUG}^{+}_{\geq}$ does not require constraint separation and re-optimization, whereas $\mathcal{NDA}^{+}_{\geq}$ does, further contributing to $\mathcal{AUG}^{+}_{\geq}$ solving instances in less time;
4. *Computational performance (impact of (VI-5))*: Concerning the algorithm that includes the separation of constraints (VI-5) in $\mathcal{NDA}^{+}_{\geq}$, the test results show that the D instances not solved by $\mathcal{NDA}^{+}_{\geq}$ are either solved when constraints (VI-5) are separated, or much better lower and upper bounds on the optimal values are obtained upon reaching the time limit, especially for $p \in \{p_7, p_8\}$. The D instances which were solved by $\mathcal{NDA}^{+}_{\geq}$ are also typically solved in less time when constraints (VI-5) are separated. However, the computational times are still not competitive with those obtained with $\mathcal{AUG}^{+}_{\geq}$. This appears to be due to the reasons mentioned in the previous point;
5. *Computational performance ($\mathcal{SANDA}^{+}_{\geq}$)*: $\mathcal{SANDA}^{+}_{\geq}$ leads to the worst results for these instances due to the poor quality lower bounds, which lead to an algorithm which explores a very large num-

Table 7
Average LP / RN gaps for $\mathcal{AUG}^{+}_{\geq}$, $\mathcal{NDA}^{+}_{\geq}$ and $\mathcal{SANDA}^{+}_{\geq}$ (D instances).

p	$\mathcal{SANDA}^{+}_{\geq}$	$\mathcal{NDA}^{+}_{\geq}$	$\mathcal{NDA}^{+}_{\geq}$ w/ (VI-5)	$\mathcal{AUG}^{+}_{\geq}$
p_5	14.96 % / 14.28 %	8.68 % / 8.83 %	1.93 % / 2.35 %	1.70 % / 1.60 %
p_6	22.37 % / 20.79 %	11.69 % / 11.77 %	1.84 % / 2.34 %	1.60 % / 1.60 %
p_7	33.37 % / 30.69 %	15.10 % / 14.53 %	1.71 % / 2.65 %	1.10 % / 1.04 %
p_8	45.75 % / 42.12 %	14.71 % / 10.03 %	1.89 % / 2.69 %	1.71 % / 1.68 %

Table 8
Average CPU times / % of unsolved instances for $\mathcal{AUG}^{+}_{\geq}$, $\mathcal{NDA}^{+}_{\geq}$ and $\mathcal{SANDA}^{+}_{\geq}$ (D instances).

p	$\mathcal{SANDA}^{+}_{\geq}$	$\mathcal{NDA}^{+}_{\geq}$	$\mathcal{NDA}^{+}_{\geq}$ w/ (VI-5)	$\mathcal{AUG}^{+}_{\geq}$
p_5	2701.8s / 75.00 %	2701.2s / 75.00 %	2701.8s / 75.00 %	1074.0s / 25.00 %
p_6	2706.2s / 75.00 %	2701.1s / 75.00 %	2700.7s / 75.00 %	556.3s / 0.00 %
p_7	3409.0s / 75.00 %	2704.0s / 75.00 %	2306.0s / 50.00 %	244.2s / 0.00 %
p_8	3600.0s / 100.00 %	2737.1s / 75.00 %	1498.5s / 25.00 %	308.5s / 0.00 %

ber of subproblems, eventually exceeding the computer's memory. In fact, most of the D instances not resolved by $\mathcal{SANDA}^{+}_{\geq}$ are due to excessive memory requirements.

In conclusion, these computational results show that, with the exception of some outliers, the new formulations provide strong lower bounds and that although the formulation on which $\mathcal{NDA}^{+}_{\geq}$ is based produces, in theory, worse lower bounds than AugNDA$_{\geq}$, $\mathcal{NDA}^{+}_{\geq}$ is far more effective than the algorithm based on AugNDA$_{\geq}$ ($\mathcal{AUG}^{+}_{\geq}$) when solving S instances. The test results for the D instances show that $\mathcal{AUG}^{+}_{\geq}$ is still the most effective algorithm for these instances. However, constraints (VI-5) have also led to improvements in the performance of the $\mathcal{NDA}^{+}_{\geq}$ algorithm, and some further insights on these inequalities and on their inclusion in a branch-and-cut algorithm may be crucial to obtain an algorithm competitive with $\mathcal{AUG}^{+}_{\geq}$ for these instances, without excessive memory requirements.

The efficacy of $\mathcal{NDA}^{+}_{\geq}$ suggests that, for S instances of the HpMP, $\mathcal{NDA}^{+}$ will perform well for instances with large values of p. We report on the results produced by this algorithm in the following section.

6.7. Computational results for the HpMP

In this section, we analyze the results obtained with the $\mathcal{NDA}^{+}$ algorithm for the HpMP, and begin by comparing the results of this algorithm with those of other algorithms known from the literature, namely, those proposed by Erdoğan et al. (2016), Bektaş et al. (2019) and Barbato and Gouveia (2024). In some tables included in the following pages, we denote these algorithms by $\mathcal{ELC}$, $\mathcal{BGS}$ and $\mathcal{BG}$ respectively. Regarding this comparison, it must be noted that the CPU times presented for those algorithms were taken from their corresponding articles, and thus, those CPU times correspond to tests run on computers with different specifications. We have used the website `https://www.cpubenchmark.net/singleCompare.php` to evaluate the CPUs of the machines used in the experiments reported in the four papers which describe the methods used for comparison. The comparison reports that the CPU used in our testing has a single-thread rating 39 % higher than the CPU used by Barbato and Gouveia (2024), 46 % higher than the CPU used by Bektaş et al. (2019) and 54 % higher than the CPU used by Erdoğan et al. (2016). In addition to this, we only compare the results for instances with $p \in \{p_3, p_4, p_5, p_6, p_8\}$, since those are the only values of p for which every algorithm has been tested.

Several tables and figures summarizing these results for the S instances are presented. Table 9 includes the average LP and RN gaps and the average CPU times obtained with $\mathcal{NDA}^{+}$ over all instances. Table 10 includes the average LP and RN gaps for each algorithm. Table 11 includes the CPU times and the percentage of unsolved instances by each algorithm (both for instances with $58 \leq n \leq 100$). Table 12 is similar to Table 11, but with results for instances with $101 \leq n \leq 400$ (for which only $\mathcal{NDA}^{+}$ and the algorithm proposed by Barbato and Gouveia (2024) were tested). Fig. 1 illustrates the number of solved instances by $\mathcal{NDA}^{+}$ and the algorithm proposed by Barbato and Gouveia (2024) in up to x s, $\forall x \in [1, 3600[$. Detailed results for every instance and algorithm compared here can be found in the supplementary material. Note that these average results only include the instances for which all algorithms were tested.

Table 9
Average LP / RN gaps, CPU times and % of unsolved instances for $\mathcal{NDA}^{+}$ (S instances).

p	LP / RN Gaps (%)	CPU times (s) / % of unsolved instances
p_1	1.28 % / 0.81 %	365.6s / 8.00 %
p_2	0.97 % / 0.44 %	203.1s / 3.13 %
p_3	0.77 % / 0.31 %	306.8s / 6.06 %
p_4	0.73 % / 0.34 %	375.9s / 6.06 %
p_5	0.87 % / 0.43 %	360.3s / 9.09 %
p_6	0.93 % / 0.34 %	342.2s / 9.09 %
p_7	1.07 % / 0.33 %	389.6s / 9.09 %
p_8	4.57 % / 3.08 %	1272.4s / 33.33 %

Table 10
Average LP / RN gaps for four algorithms (S instances, $58 \leq n \leq 100$).

	$\mathcal{NDA}^{+}$ Gaps (%)		$\mathcal{BG}$ Gaps (%)		$\mathcal{BGS}$	$\mathcal{ELC}$
p	LP	RN	LP	RN	LP	RN
p_3	0.96 %	0.39 %	1.66 %	0.41 %	1.51 %	0.75 %
p_4	0.97 %	0.44 %	1.49 %	0.34 %	1.32 %	0.48 %
p_5	1.16 %	0.54 %	2.03 %	0.88 %	1.42 %	0.49 %
p_6	1.18 %	0.29 %	2.83 %	1.66 %	1.57 %	0.45 %
p_8	3.19 %	1.85 %	6.16 %	3.35 %	5.68 %	3.23 %

Table 11
Average CPU times / % of unsolved instances for four algorithms (S instances, $58 \leq n \leq 100$).

p	$\mathcal{NDA}^{+}$	$\mathcal{BG}$	$\mathcal{BGS}$	$\mathcal{ELC}$
p_3	4.6s / 0.00 %	1.2s / 0.00 %	59.8s / 0.00 %	468.4s / 0.00 %
p_4	5.9s / 0.00 %	37.1s / 0.00 %	47.3s / 0.00 %	492.9s / 9.09 %
p_5	15.8s / 0.00 %	361.5s / 9.09 %	60.5s / 0.00 %	752.5s / 18.18 %
p_6	6.7s / 0.00 %	788.6s / 18.18 %	90.2s / 0.00 %	419.2s / 9.09 %
p_8	120.7s / 0.00 %	1956.8s / 36.36 %	2898.7s / 72.73 %	2519.7s / 54.55 %

Some remarks are in order:

1. *LP relaxation bounds*: Better LP relaxation bounds are obtained with $\mathcal{NDA}^{+}$ than with the algorithms proposed by Barbato and Gouveia (2024) and Bektaş et al. (2019);
2. *RN relaxation bounds*: For $p = p_3$, the RN relaxation bounds provided by $\mathcal{NDA}^{+}$ are close to or better than those provided by other al-

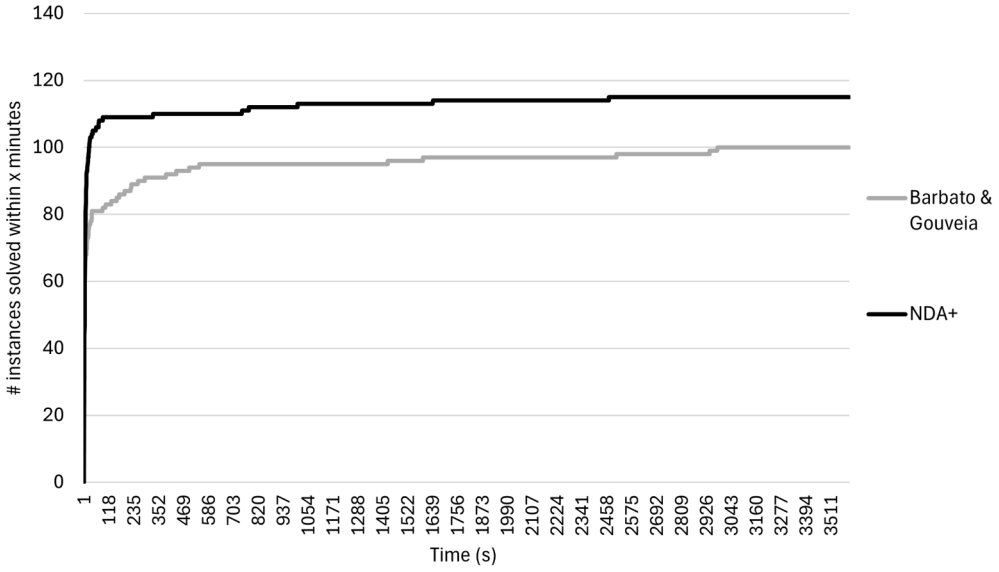


Fig. 1. Number of solved S instances of the benchmark set within fixed time thresholds for two algorithms.

Table 12
Average CPU times / % of unsolved instances for two algorithms (S instances, $101 \leq n \leq 400$).

p	$\mathcal{NDA}^+$	$\mathcal{BG}$
p_3	399.0s / 0.00 %	1468.7s / 28.57 %
p_4	978.8s / 14.29 %	1162.6s / 28.57 %
p_5	1039.4s / 28.57 %	2072.3s / 57.14 %
p_6	1042.8s / 28.57 %	3086.5s / 85.71 %
p_8	2579.3s / 71.43 %	2335.3s / 57.14 %

gorithms. For $p = p_4$ and $p = p_5$, the RN relaxation bounds provided by Barbato & Gouveia's algorithm (Barbato & Gouveia, 2024) and the lower bounds provided by Erdoğan et al. (2016) are slightly better respectively. Finally, for $p = p_6$, $\mathcal{NDA}^+$ provides slightly better RN relaxation bounds, and for $p = p_8$, these differences become very considerable, emphasizing the effectiveness of constraints (VI-3) for instances with large values of p (as noted in Section 6.5);

3. *Computational performance (overview)*: In general, $\mathcal{NDA}^+$ requires less CPU time to solve benchmark instances of the HpMP than the algorithms tested by Erdoğan et al. (2016), Bektaş et al. (2019) and Barbato and Gouveia (2024);
4. *Computational performance (comparing $\mathcal{NDA}^+$ and $\mathcal{BG}$, part 1)*: The majority of S instances are solved by $\mathcal{NDA}^+$ to optimality within 1 min (79.22 %, or 202/255 instances). On this topic, it is also interesting to analyze Fig. 1, since it shows that, out of the 125 S instances for which both $\mathcal{NDA}^+$ and $\mathcal{BG}$ were tested, the majority are either solved by $\mathcal{NDA}^+$ in up to 1 min (106/125) or are not solved within the time limit (10/125). Only a few instances (9/125) are solved in between 1 min and 1 h. A similar trend holds for the instances for which only $\mathcal{NDA}^+$ was tested;
5. *Computational performance ($p = p_8$)*: However, as noted when analyzing the results obtained for the HpMP$_{\geq}$, instances with $p = p_8$ remain some of the most difficult. Although $\mathcal{NDA}^+$ has solved all of these instances with $n \leq 100$, many larger instances with $p = p_8$ remain unsolved;
6. *Instances solved for the first time*: To the best of our knowledge, some instances reported in the literature have not been solved to proven optimality before. However, some of those have been solved by $\mathcal{NDA}^+$. This includes the instances based on kroB100, kroD100 and rd100 with $p = 33$, u159 with $p = 31$, rat195 with $p = 27, 39, 48$, d198 with $p = 19, 49$ and pr226 with $p = 22, 32, 56$ (this comparison also takes into consideration the results obtained by Marzouk et al. (2016));
7. *Low optimality gaps for unsolved instances*: Furthermore, in S instances which have not been solved by $\mathcal{NDA}^+$ within the time limit, the optimality gaps are often below 2 %;
8. *Computational performance (comparing $\mathcal{NDA}^+$ and $\mathcal{BG}$, part 2)*: For 40 % of the instances based on gil262 and rd400 on which both $\mathcal{NDA}^+$ and the algorithm proposed by Barbato and Gouveia (2024) were tested, the latter gave considerably better CPU times. These results appear to be due to the fact that the values of p defining those instances are close to p^*, and thus, it is likely the formulations on which both algorithms are based give similar LP and RN relaxation bounds, and thus, the algorithm proposed by Barbato and Gouveia (2024) (based on a formulation with fewer variables than the formulation on which $\mathcal{NDA}^+$ is based) has better computational performance. This motivates the study of formulations with stronger constraints to prevent solutions with less than p cycles and defined on the space of variables u_{ij} and y_i, as mentioned in Section 7.2.

We describe next the results for the more difficult D instances. Table 13 includes the average computational times and percentage of unsolved instances for all values of p. Table 14 includes the average data for the two variants of $\mathcal{NDA}^+$ (with and without constraints (VI-5)) and for $\mathcal{BG}$. $\mathcal{BG}$ was also applied to the new instances in a machine with the same specifications, settings and version of CPLEX as the one used in Barbato and Gouveia (2024).

1. *Lower bounds*: $\mathcal{NDA}^+$ only provides strong LP relaxation bounds for small values of p, but, for high values of p, these lower bounds can be substantially improved if constraints (VI-5) are also considered. Concerning the RN relaxation bounds, these show that, when solving the D instances, CPLEX's cut generation algorithms are seldom capable of considerably improving lower bounds if constraints (VI-5) are separated. In contrast, if constraints (VI-5) are not separated, these cut generation algorithms are capable of improving the lower bounds, but only for very high values of p;
2. *Computational performance (overview for $\mathcal{NDA}^+$)*: In contrast with its performance for the S instances with $n \leq 200$, $\mathcal{NDA}^+$ consistently solves D instances only with $n \leq 50$ or with very low values of p. Furthermore, instances with very high values of p can be solved more consistently by $\mathcal{NDA}^+$ when constraints (VI-5) are also considered in the algorithm;

Table 13
Average LP / RN gaps and average CPU times and % of unsolved instances for $\mathcal{NDA}^+$, with and without (VI-5) (D instances).

	$\mathcal{NDA}^+$		$\mathcal{NDA}^+$ w/ (VI-5)	
p	LP / RN Gaps (%)	CPU times (s) / % unsolved	LP / RN Gaps (%)	CPU times (s) / % unsolved
p_1	0.33 % / 0.33 %	8.5s / 0.00 %	0.33 % / 0.33 %	7.6s / 0.00 %
p_2	0.58 % / 0.70 %	52.2s / 0.00 %	0.50 % / 0.50 %	42.1s / 0.00 %
p_3	2.18 % / 2.21 %	2130.7s / 50.00 %	1.39 % / 1.39 %	2127.8s / 50.00 %
p_4	4.35 % / 4.45 %	2700.9s / 75.00 %	1.92 % / 1.97 %	2701.9s / 75.00 %
p_5	8.68 % / 8.83 %	2701.2s / 75.00 %	1.94 % / 2.21 %	2701.9s / 75.00 %
p_6	11.69 % / 11.68 %	2700.8s / 75.00 %	1.82 % / 2.19 %	2700.7s / 75.00 %
p_7	15.11 % / 14.56 %	2704.1s / 75.00 %	1.70 % / 2.25 %	2305.6s / 50.00 %
p_8	14.70 % / 10.11 %	2738.2s / 75.00 %	1.86 % / 2.47 %	1496.3s / 25.00 %

Table 14
Average CPU times / % of unsolved instances for three algorithms (D instances).

p	$\mathcal{NDA}^+$	$\mathcal{NDA}^+$ w/ (VI-5)	$\mathcal{BG}$
p_3	2130.7s / 50.00 %	2127.8s / 50.00 %	1952.8s / 50.00 %
p_4	2700.9s / 75.00 %	2701.9s / 75.00 %	2700.6s / 75.00 %
p_5	2701.2s / 75.00 %	2701.9s / 75.00 %	2709.8s / 75.00 %
p_6	2700.8s / 75.00 %	2700.7s / 75.00 %	3316.9s / 75.00 %
p_8	2738.2s / 75.00 %	1496.3s / 25.00 %	3600.0s / 100.00 %

3. *Optimality gaps for unsolved instances*: Unlike what happens with the S instances, in the D instances that have not been solved by $\mathcal{NDA}^+$ within the time limit, the optimality gaps are often quite high. These gaps are considerably improved when the (VI-5) constraints are considered, but optimality gaps can still reach 10 % in some of these instances;
4. *Computational performance (comparing $\mathcal{NDA}^+$ and $\mathcal{BG}$)*: The results given by $\mathcal{BG}$ also emphasize the difficulty of the D instances. Instances with high values of p are more difficult than instances with intermediate or low values of p. These results are consistent with results reported in the literature (e.g. Erdoğan et al., 2016, Bektaş et al., 2019, Barbato et al., 2023 for the S instances). $\mathcal{NDA}^+$ with or without (VI-5) performs better than $\mathcal{BG}$ for the D instances, especially for high values of p. For instances not solved by the two algorithms, the optimality gaps obtained with $\mathcal{NDA}^+$ with (VI-5) are also considerably smaller than the optimality gaps obtained with $\mathcal{BG}$;
5. *Computational performance ($p = p_8$)*: Unlike what happens for the S instances, for the algorithms proposed in this work, the D instances with $p = p_8$ are easier than the D instances with slightly lower values of p. This is illustrated by the fact that $\mathcal{NDA}^+$ with constraints (VI-5) was not able to solve several of these instances with $p = p_6$, but was able to solve most of the instances with $p = p_8$.

In conclusion, for the S instances, the results obtained with $\mathcal{NDA}^+$ for the HpMP show that it is capable of solving several instances which had not yet been solved and provides strong lower bounds for all values of p. Moreover, it is competitive with the current state-of-the-art in solving S instances with up to 400 nodes. In contrast, the results also show that the D instances are much more difficult. D instances with only 100 nodes prove to be quite difficult for any algorithm studied in this work (other than $\mathcal{AUG}^+_{\geq}$, for the HpMP$_{\geq}$). However, for these instances, $\mathcal{NDA}^+$ is also very competitive with the algorithm proposed by Barbato and Gouveia (2024), producing considerably better lower bounds, especially when constraints (VI-5) are also considered.

6.8. Computational results for the HpMP (including setup costs)

The results of some computational tests for the variant of the HpMP with setup costs are also reported to assess how the performance of the $\mathcal{NDA}^+$ algorithm is influenced by setup costs of different magnitudes. From each instance for the HpMP with $n \leq 200$, two different instances (labeled X-F2 and X-F1 respectively, where X is the designation of the original instance) for this variant were created, with integer setup costs of varying magnitudes relative to the optimal value v^* of the corresponding 2-factor relaxation. In particular, if the instance name terminates in:

- F2, the setup costs were generated following a uniform distribution in the range $[v^*/2, 3v^*/2]$;
- F1, the setup costs were generated following a uniform distribution in the range $[v^*/50, 3v^*/50]$;

Due to what was mentioned in Section 5, we can still use the SB strategy in the formulations to solve these instances by sorting the nodes by non-descending order of their setup costs. In contrast with the previous tests, these tests do not incorporate a primal heuristic since the heuristic described in Section 5.3 of the manuscript does not account for setup costs.

We begin by analyzing the results obtained for the S instances. Table 15 includes the average gaps and CPU times for $\mathcal{NDA}^+$ for these instances. Note that F0 refers to instances with no setup costs.

Concerning the computational results, some of the conclusions drawn for instances with no setup costs hold for this case (for example, CPU times worsen when $p = p_8$), although new conclusions can be drawn. As far as we know, this is the first study on this variant, and thus, we focus our analysis on how the lower bounds and computational performance are affected by the magnitude of the setup costs:

1. *Lower bounds*: The LP and RN gaps tend to be smaller for instances with high setup costs than they are for instances with low setup costs. This is because as the setup costs increase, the difference in absolute terms between the lower bound and the optimal value remains similar, and since the optimal value increases, this leads to a decrease in the gaps. This is also why, for the instances with setup costs, the gaps tend to decrease as p increases, in contrast to what happens for instances with no setup costs. Since p is larger, there are more depots, and hence the optimal value is higher due to the setup costs associated with those additional depots, thus lowering gaps;
2. *Computational performance*: Although the gaps decrease as the setup costs increase, the CPU times do not. For all values of p other than p_7 and p_8, instances with high setup costs appear to be more difficult than instances with low setup costs. This difficulty is also reflected in the percentage of unsolved F2 instances with intermediate values of p, which is much greater than the corresponding percentage of unsolved F0 instances. In contrast, instances with $p \in \{p_7, p_8\}$ are solved in less time when the setup costs are high.

We also study the performance of the algorithm presented in this work when solving D instances. Tables 16 and 17 include the average gaps and CPU times required by $\mathcal{NDA}^+$ to solve these instances, including and not including the separation of constraints (VI-5).

From these results, we see that for the D instances, the lower bounds are influenced by the setup costs similarly to how they are influenced by the setup costs for the S instances. However, there is one new finding related to computational performance. The average times show that,

Table 15
Average gaps, CPU times and % of unsolved S instances with setup costs for $\mathcal{NDA}^+$.

	F2			F1			F0		
p	LP	RN	CPU / % unsolved	LP	RN	CPU / % unsolved	LP	RN	CPU / % unsolved
p_1	0.43 %	0.23 %	339.5s / 5.00 %	1.12 %	0.52 %	219.1s / 5.00 %	1.04 %	0.56 %	28.0s / 0.00 %
p_2	0.30 %	0.15 %	413.1s / 7.41 %	1.06 %	0.43 %	317.8s / 3.70 %	0.95 %	0.40 %	17.4s / 0.00 %
p_3	0.33 %	0.21 %	969.0s / 17.86 %	1.17 %	0.57 %	856.2s / 14.29 %	0.88 %	0.42 %	245.2s / 3.57 %
p_4	0.38 %	0.28 %	1011.0s / 17.86 %	1.52 %	0.90 %	1018.5s / 14.29 %	0.89 %	0.52 %	293.7s / 3.57 %
p_5	0.23 %	0.16 %	1013.4s / 17.86 %	1.27 %	0.64 %	641.6s / 7.14 %	1.12 %	0.70 %	325.9s / 3.57 %
p_6	0.18 %	0.12 %	673.5s / 10.71 %	1.11 %	0.44 %	569.8s / 10.71 %	1.21 %	0.60 %	203.3s / 3.57 %
p_7	0.13 %	0.09 %	220.7s / 3.57 %	0.92 %	0.36 %	420.7s / 7.14 %	1.61 %	0.80 %	404.7s / 7.14 %
p_8	0.19 %	0.20 %	1017.6s / 17.86 %	2.35 %	1.63 %	1170.3s / 21.43 %	4.58 %	3.19 %	1314.4s / 17.86 %

Table 16
Average gaps, CPU times and % of unsolved D instances with setup costs for $\mathcal{NDA}^+$.

	F2			F1			F0		
p	LP	RN	CPU / % unsolved	LP	RN	CPU / % unsolved	LP	RN	CPU / % unsolved
p_1	0.15 %	0.10 %	4.8s / 0.00 %	0.27 %	0.28 %	10.8s / 0.00 %	0.33 %	0.33 %	15.4s / 0.00 %
p_2	0.23 %	0.23 %	927.4s / 25.00 %	0.59 %	0.60 %	30.9s / 0.00 %	0.58 %	0.70 %	29.1s / 0.00 %
p_3	0.52 %	0.54 %	1101.3s / 25.00 %	1.99 %	2.02 %	1865.2s / 50.00 %	2.18 %	2.21 %	1922.9s / 50.00 %
p_4	1.41 %	1.41 %	2700.4s / 75.00 %	5.30 %	5.13 %	2700.5s / 75.00 %	4.35 %	4.45 %	2701.9s / 75.00 %
p_5	1.02 %	0.94 %	2704.4s / 75.00 %	7.00 %	5.54 %	2701.8s / 75.00 %	8.68 %	8.83 %	2700.7s / 75.00 %
p_6	1.12 %	1.08 %	2700.5s / 75.00 %	8.89 %	6.62 %	2700.3s / 75.00 %	11.69 %	11.68 %	2701.0s / 75.00 %
p_7	1.21 %	1.27 %	2549.9s / 50.00 %	10.67 %	7.82 %	2708.5s / 75.00 %	15.11 %	14.56 %	2701.8s / 75.00 %
p_8	0.88 %	1.20 %	2065.8s / 50.00 %	10.08 %	8.42 %	2723.5s / 75.00 %	14.70 %	10.11 %	2730.1s / 75.00 %

Table 17
Average gaps, CPU times and % of unsolved D instances with setup costs for $\mathcal{NDA}^+$ with (VI-5).

	F2			F1			F0		
p	LP	RN	CPU / % unsolved	LP	RN	CPU / % unsolved	LP	RN	CPU / % unsolved
p_1	0.15 %	0.10 %	5.1s / 0.00 %	0.27 %	0.27 %	1162.9s / 0.00 %	0.33 %	0.33 %	5.0s / 0.00 %
p_2	0.20 %	0.21 %	938.4s / 25.00 %	0.53 %	0.49 %	26.5s / 0.00 %	0.50 %	0.50 %	42.1s / 0.00 %
p_3	0.44 %	0.44 %	1031.9s / 25.00 %	1.66 %	1.49 %	1906.7s / 50.00 %	1.39 %	1.39 %	2269.3s / 50.00 %
p_4	1.19 %	1.19 %	2102.8s / 50.00 %	3.88 %	3.84 %	2700.7s / 75.00 %	1.92 %	1.97 %	2700.7s / 75.00 %
p_5	0.51 %	0.50 %	1767s / 25.00 %	2.07 %	1.99 %	2701.3s / 75.00 %	1.94 %	2.21 %	2700.6s / 75.00 %
p_6	0.55 %	0.56 %	1938.5s / 50.00 %	1.92 %	1.79 %	2173.0s / 50.00 %	1.82 %	2.19 %	2700.2s / 75.00 %
p_7	0.52 %	0.53 %	1501.9s / 25.00 %	1.42 %	1.38 %	2129.1s / 50.00 %	1.70 %	2.25 %	2155.8s / 50.00 %
p_8	0.40 %	0.41 %	1099.1s / 25.00 %	2.03 %	1.93 %	1967.1s / 50.00 %	1.86 %	2.47 %	1317.9s / 25.00 %

similar to what was observed for the S instances, instances with high setup costs are more difficult than the corresponding instances with low setup costs when $p \in \{p_1, p_2\}$. Moreover, for large values of p, instances with high setup costs are solved in considerably less time than instances with low or no setup costs when p is large. This is so far similar to the conclusions drawn for the S instances, but appears to be more noticeable for the D instances, since most of the D instances with very high setup costs are solved by $\mathcal{NDA}^+$ with constraints (VI-5) (even for intermediate values of p). This is not the case when there are no setup costs.

7. Final remarks

7.1. Conclusions

We have studied location-routing based formulations for the HpMP and the HpMP$_{\geq}$. We have proposed new exponentially sized sets of constraints to prevent solutions with less than p cycles defined using only edge and depot variables, and have established relations between some of those inequalities and known formulations defined using edge and NDA variables. Furthermore, we have proposed new compact formulations including EDA variables. Projection results relating the new formulations with other formulations defined only with the edge and NDA variables have been stated and proven, and as a consequence, new sets of valid inequalities including edge and NDA variables have been proposed. Computational testing has shown that the strongest formulation with EDA variables provides very strong LP relaxation bounds, especially for large values of p, and that it is effective in solving instances with p close to $\lfloor n/3 \rfloor$ and $n \leq 100$. Further computational testing has demonstrated the effectiveness of a branch-and-cut algorithm based on an exponentially sized formulation that includes some of the new valid inequalities, as it can solve instances of the HpMP with up to 400 nodes, many of which are solved in less time than current exact algorithms require for this problem.

Our research also led to the development of new instances characterized by a high percentage of triples $(i, j, k) : i < j,\ k \neq i, j$ resulting in a violated triangle inequality. Our computational results suggest these instances are more challenging than the traditional benchmark instances that have been studied for years.

As a concluding comment, we emphasize that the use of the SB strategy has a strong impact on the efficiency of the proposed methods. Thus, a question worth considering concerns the applicability of this or related strategies to variants of this problem. We have therefore also addressed two variants of the HpMP: one, with node setup costs (i.e., there exists a cost associated with designating a certain node as a depot) and which illustrates a situation in which the proposed SB strategy is still applicable (although not in a straightforward way), and another that illustrates a situation where it is not (or at least, it is not clear how to include an SB strategy unless the costs of the variables in the objective function are structured in some very specific way).

7.2. Future work

Future work will focus on addressing some of the open questions identified throughout this paper.

As mentioned before, the main topic of the current paper is motivated by the fact that the knowledge on inequalities that prevent solutions with fewer than p cycles is not (yet) comparable with what is known regarding inequalities that prevent solutions with more than p cycles. Our research has led to new and competitive formulations (compared to the state of the art). However, and following the alternative study on inequalities preventing solutions with more than p cycles (see Canas & Gouveia, 2025), we believe that it is worth obtaining inequalities using only the u_{ij} and y_i variables of "similar" strength to the clique-based inequalities (wCC≤) and (CC≤) described in Section 3.1, perhaps generalizing inequalities (PI≥)). This is one target topic of research.

A closely related topic is to go a step further and study the projection of the set of feasible solutions of the AugNDA$_{\geq L}$ formulation in the space of the u_{ij} variables alone. This description would make it easier to establish a theoretical comparison between the models presented here and the inequalities described by Barbato and Gouveia (2024). From a theoretical point of view, it is easy to show that the LP relaxation of the strongest formulation here presented (AugNDA$_{\geq}$) does not imply the Quasi-Hamiltonian cycle (QHC) inequalities presented by Barbato and Gouveia (2024), suggesting that a combined approach could be worth using. Conversely, our computational results also show that the reverse implication does not hold, which makes finding this projected polyhedron even more relevant and interesting.

One other topic of research is to look at the description of PROJ$_{u,h}$(ADM(AugNDA$_{\geq L}$)). As noted earlier, it is clear that there exist additional sets of valid inequalities implied by AugNDA$_{\geq L}$ that have not been mentioned in this paper. However, it is not clear whether these constraints are computationally relevant, since the constraints (VI-5) already appear to provide lower bounds comparable to those obtained with AugNDA$_{\geq L}$. A final related topic is the more effective computational use of constraints (VI-5), which are crucial to obtain strong lower bounds in certain types of instances. However, if not handled carefully, they can significantly slow down a branch-and-cut algorithm. This is worth mentioning because $\mathcal{NDA}^{+}$ (and $\mathcal{BG}$) can solve several benchmark instances with up to 400 nodes, yet hardly solves most of the randomly generated instances with more than 75 nodes in one hour, although the inclusion of the (VI-5) constraints significantly improves lower bounds for these instances, bringing them close to those achieved by $\mathcal{AUG}^{+}_{\geq}$.

CRediT authorship contribution statement

Francisco Canas: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Luís Gouveia:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

None.

Acknowledgements

The authors are thankful to the reviewers for the feedback which helped improve this paper, and to Michele Barbato for testing the $\mathcal{BG}$ algorithm for the D instances. L. Gouveia and F. Canas were supported by Portuguese National Funding from FCT - Fundação para a Ciência e a Tecnologia under projects UIDB/04561/2020 (https://doi.org/10.54499/UIDB/04561/2020) and UIDP/04561/2020 (https://doi.org/10.54499/UIDP/04561/2020). F. Canas was supported by a PhD Studentship from FCT - Fundação para a Ciência e a Tecnologia (reference: 2023.00825.BD).

Supplementary material

Supplementary material associated with this article can be found, in the online version, at 10.1016/j.ejor.2025.10.023

References

Applegate, D. L., Bixby, R. E., Chvatál, V., & Cook, W. J. (2006). The traveling salesman problem: A computational study. Princeton University Press.

Barbato, M., Canas, F., Gouveia, L., & Pesneau, P. (2023). Node based compact formulations for the Hamiltonian p-median problem. *Networks*, *82* (4), 336–370.

Barbato, M., & Gouveia, L. (2024). The Hamiltonian p-median problem: Polyhedral results and branch-and-cut algorithms. *European Journal of Operational Research*, *316* (2), 473–487.

Bektaş, T., Gouveia, L., & Santos, D. (2019). Revisiting the Hamiltonian p-median problem: A new formulation on directed graphs and a branch-and-cut algorithm. *European Journal of Operational Research*, *276* (1), 40–64.

Bektaş, T., Gouveia, L., & Santos, D. (2020). Compact formulations for multi-depot routing problems: Theoretical and computational comparisons. *Computers & Operations Research*, *124*, 105084. https://www.sciencedirect.com/science/article/pii/S030505482030201X. https://doi.org/https://doi.org/10.1016/j.cor.2020.105084

Belenguer, J.-M., Benavent, E., Prins, C., Prodhon, C., & Wolfler Calvo, R. (2011). A branch-and-cut method for the capacitated location-routing problem. *Computers & Operations Research*, *38* (6), 931–941.

Branco, I. M., & Coelho, J. D. (1990). The Hamiltonian p-median problem. *European Journal of Operational Research*, *47* (1), 86–95.

Canas, F., & Gouveia, L. (2025). Formulations and branch-and-cut algorithms for cycle covers with up to p cycles. *European Journal of Operational Research*. *327* (1), 42–57.

Catanzaro, D., Gouveia, L., & Labbé, M. (2015). Improved integer linear programming formulations for the job sequencing and tool switching problem. *European Journal of Operational Research*, *244* (3), 766–777. https://doi.org/https://doi.org/10.1016/j.ejor.2015.02.018

Cornuéjols, G., & Pulleyblank, W. R. (1980). Perfect triangle-free 2-matchings. In *Combinatorial Optimization II* , pp. 1–7). Berlin, Heidelberg: Springer Berlin Heidelberg. https://doi.org/10.1007/BFb0120901. https://doi.org/10.1007/BFb0120901

Erdoğan, G., Laporte, G., & Rodríguez-Chía, A. M. (2016). Exact and heuristic algorithms for the Hamiltonian p-median problem. *European Journal of Operational Research*, *253* (2), 280–289.

Garey, M. R., & Johnson, D. S. (1979). Computers and intractability: A guide to the theory of NP-completeness. New York, NY, USA: W. H. Freeman & Co.

Gavish, B., & Graves, S. C. (1978). The travelling salesman problem and related problems. *Working Paper OR-078-78; Operations Research Center, MIT: Cambridge, MA, USA*.

Glaab, H., & Pott, A. (2000). The Hamiltonian p–median problem. *The Electronic Journal of Combinatorics*, *7* (1), R42.

Gollowitzer, S., Gouveia, L., Laporte, G., Pereira, D. L., & Wojciechowski, A. (2014). A comparison of several models for the Hamiltonian p-median problem. *Networks*, *63* (4), 350–363.

Gollowitzer, S., Pereira, D. L., & Wojciechowski, A. (2011). New models for and numerical tests of the Hamiltonian p-median problem. In J. Pahl, T. Reiners, & S. Voß (Eds.), *Network optimization* (pp. 385–394). Springer Berlin Heidelberg.

He, P., Hao, J.-K., & Wu, Q. (2024). A hybrid genetic algorithm for the Hamiltonian p-median problem. *Networks*, *83* (2), 348–367.

Laporte, G., Nobert, Y., & Pelletier, P. (1983). Hamiltonian location problems. *European Journal of Operational Research*, *12* (1), 82–89.

Marzouk, A. M., Moreno-Centeno, E., & Üster, H. (2016). A branch-and-price algorithm for solving the Hamiltonian p-median problem. *INFORMS Journal on Computing*, *28* (4), 674–686.

Miller, C. E., Tucker, A. W., & Zemlin, R. A. (1960). Integer programming formulation of traveling salesman problems. *Journal of the ACM*, *7* (4), 326–329. https://doi.org/10.1145/321043.321046

Reinelt, G. (1991). TSPLIB - A traveling salesman problem library. *ORSA Journal on Computing*, *3* (4), 376–384. https://doi.org/10.1287/ijoc.3.4.376. https://doi.org/10.1287/ijoc.3.4.376

Wolsey, L. (2020). Integer programming. , pp. 148). John Wiley & Sons, Ltd.